

RCFuzzer: Recommendation-based Collaborative Fuzzer

Hyeonmin Mo, Jongmun Yang

*Department of Computer and Software (Automotive-Computer Convergence),
Hanyang University, Seoul, Republic of Korea*

Yunho Kim*

Department of Computer Science, Hanyang University, Seoul, Republic of Korea

Abstract

Fuzzing is an effective technique for detecting bugs by executing programs with randomly generated or mutated inputs. However, as various fuzzers have been developed, selecting the most suitable fuzzer for a specific program has become increasingly difficult. To address this issue, collaborative fuzzing techniques have been proposed, which combine multiple fuzzers and select the optimal one. However, existing approaches are inefficient and have limited accuracy, as they require significant time to evaluate fuzzer performance and fail to effectively utilize the latest results from the fuzzing campaign.

To overcome these challenges, we propose RCFuzzer, a ReCommendation based collaborative Fuzzer. RCFuzzer treats the fuzzer selection problem as a Multi-Armed Bandit(MAB) problem and improves the efficiency and accuracy of selecting the optimal fuzzer using Thompson sampling. First, RCFuzzer is efficient because it directly utilizes the current fuzzing results, eliminating the need for additional time to evaluate individual fuzzers. Second, RCFuzzer can accurately select the optimal fuzzer by using the fuzzing results obtained from the current state of the fuzzing target as feedback. Additionally, to further improve the accuracy of fuzzer selection, RCFuzzer adopts the branch difficulty heuristics, which assigns different weights to branches based on their difficulty to cover and evaluates fuzzers accordingly.

The empirical evaluation on the 47 programs from MAGMA, UNIFUZZ and Google's Fuzzer-Test-Suite shows that RCFuzzer outperforms individual fuzzers in code coverage and crash detection capability. Additionally, RCFuzzer achieves higher code coverage for 29 out of 47 programs and detects 18 more unique crashes than autofz, the state-of-the-art collaborative fuzzer.

*Corresponding author

Email addresses: hyeonminmo@hanyang.ac.kr (Hyeonmin Mo),
jongmunyang@hanyang.ac.kr (Jongmun Yang), yunhokim@hanyang.ac.kr (Yunho Kim)

Keywords: Collaborative fuzzing, Crash detection, Thompson sampling

1. INTRODUCTION

Fuzzing (Boehme et al., 2021; Manès et al., 2021; Miller et al., 1990; Fioraldi et al., 2023; Holler et al., 2012) is one of the most effective technique to find software bugs. Fuzzing provides random or mutated inputs to a program, repeatedly executes it, collects feedback, and monitors for abnormal behavior (e.g., crashes or unexpected results). Various techniques have been proposed to improve fuzzing performance, and fuzzing has established itself as a critical tool for discovering defects in commercial software.

However, as fuzzers advance and new techniques emerge, the process of selecting the most suitable fuzzer for a specific program has become increasingly complex. Since each program has different structures, inputs, and characteristics, a fuzzer that works well on one program may be ineffective for another. For instance, Google’s fuzzbench experiment results have shown that MOpt (Lyu et al., 2019) achieved the highest edge coverage for the `libxml2` program, while QSYM (Yun et al., 2018) is the most effective in achieving the highest edge coverage (Google, 2020) for the `sqlite3` program. This emphasizes that relying on only one fuzzer can result in different performance across programs, leading to ineffective testing.

To address this issue, the collaborative fuzzing approaches that orchestrate various individual fuzzers have been proposed. The collaborative fuzzing approaches evaluate the individual fuzzers’ performance and schedules the optimal fuzzer for the current state of the target program. By using the optimal input generation methods and input scheduling strategies for the different states, collaborative fuzzing can detect more bugs than individual fuzzers. However, the existing collaborative fuzzing approaches (Chen et al., 2019; Güler et al., 2020; Fu et al., 2023) have limitations that lead to inefficiency and ineffectiveness. First, the existing collaborative fuzzing requires significant amount of time to evaluate the individual fuzzers. For example, CUPID (Güler et al., 2020) requires past performance data of the individual fuzzers needs to execute all the individual fuzzers in each iteration to evaluate the performance of fuzzers. Second, the accuracy in selecting the optimal fuzzer is low because the performance evaluation is not based on the current state of the fuzzing target. ENFUZZ (Chen et al., 2019) does not select the optimal fuzzer, but just uses all the given fuzzers even if the fuzzers are not suitable for the target. CUPID tries to select the optimal fuzzer, but the evaluation of fuzzer is based on the past performance data obtained from the fuzzing campaigns for another fuzzing targets. autofz (Fu et al., 2023) does not synchronize the seed inputs used in the preparation phase and the focus phase so the expectations in the preparation phase may not meet in the focus phase.

To overcome these limitations, we propose RCFuzzer (ReCommendation-based collaborative Fuzzer). RCFuzzer aims to maximize fuzzing efficiency by

dynamically selecting fuzzers using recommendation approach. RCFuzzer models the fuzzer selection process as a Multi-Armed Bandit (MAB) problem (Sutton and Barto, 2018; Thompson, 1933). The MAB problem aims to maximize cumulative rewards by allocating limited resources to multiple slot machines. Each fuzzer is treated as a slot machine, and the goal is to decide which fuzzer to allocate resources to. RCFuzzer addresses this problem by employing Thompson sampling, which has demonstrated superior performance (Chapelle and Li, 2011; Komiyama et al., 2019). Thompson sampling is a probabilistic algorithm that dynamically selects fuzzers based on their success and failure counts, balancing exploration and exploitation to adapt to changing program conditions in run-time.

The empirical evaluation on 47 real-world programs from MAGMA (Hazimeh et al., 2020), UNIFUZZ (Li et al., 2021) and Fuzzer-test-suite from Google (Google, 2021) showed that RCFuzzer outperformed the individual fuzzers and the state-of-the-art collaborative fuzzer in both code coverage and crash detection capability. Compared to individual fuzzers, RCFuzzer achieved the highest ranking in average code coverage with a score of 1.4. RCFuzzer also found 4 more unique crashes that no individual fuzzer could detect, demonstrating its superiority in exploring diverse code paths. Compared to autofz, RCFuzzer achieved higher code coverage for 29 out of 47 programs and detected 30 additional unique crashes missed by autofz, highlighting the strengths of collaborative fuzzing approach used by RCFuzzer.

The main contributions of this paper are as follows:

- **We propose a novel recommendation-based collaborative fuzzer for dynamic fuzzer selection:** RCFuzzer maximizes fuzzing effectiveness by using a recommendation algorithm that dynamically selects fuzzers. This approach is more flexible and adaptive than previous methods based on manual combinations or past performance data of the individual fuzzers reliance. (Section 3.3.1)
- **We have designed the branch difficulty-based heuristics to improve the effectiveness of fuzzer selection:** RCFuzzer addresses the performance distortion issues that arise in Thompson sampling-based fuzzer selection by using the branch-based heuristics. First, to solve the problem of fuzzers being overestimated in early stage, the collection phase was added to collect and analyze the performance data of various fuzzers, allowing the difficulty of the program code to cover to be reflected more accurately. Second, to resolve the issue of fuzzers that cover hard-to-cover code being underestimated, we implemented the heuristic to give more weights to the fuzzer that covers hard-to-cover code. By combining these two approaches, RCFuzzer improves the effectiveness of fuzzing that uses thompson sampling and achieves higher code coverage and bug detection performance across diverse programs. (see Section 3.2 and Section 3.3.2)
- **We evaluate RCFuzzer on 47 real-world programs and show its effectiveness:** RCFuzzer was evaluated on 47 programs from MAGMA,

UNIFUZZ and Fuzzer-test-suite from Google, demonstrating superior performance in terms of code coverage and unique crashes detected compared to individual fuzzers and the state-of-the-art collaborative fuzzing technique, autofz. In particular, RCFuzzer detected 30 additional unique crashes missed by autofz and achieved higher coverage in 29 out of 47 programs. (see Section 5)

- **We open-source to foster further research in this area:** <https://github.com/hyeonminmo/RCFuzzer>

The rest of the sections are organized as follows. Section 2 explains the background knowledge to understand RCFuzzer. Section 3 describes the technical details of RCFuzzer. Section 4 and Section 5 shows experiment setup and results, respectively. Section 6 discusses observations through the experiments. Section 7 discusses the related work. Section 8 summarizes this article with future work.

2. BACKGROUND

2.1. Fuzzing

Fuzzing is an automated software testing technique that generates a large number of randomly generated inputs. To improve the effectiveness, modern fuzzing techniques use code coverage feedback combined with mutational test generation, enabling the fuzzer to create new inputs based on the coverage information and explore uncovered parts of code.

Algorithm 1 presents the algorithm of fuzzing. It takes as inputs a target program and a set of initial seed inputs, which are valid inputs for the target program. In line 2, S and Crash is initialized as S_{init} and an empty set $\emptyset$, respectively. From lines 3 to 7, the algorithm iterates the fuzzing loop to generate new inputs and executes the target program with the newly generated inputs until timeout occurs. First, in line 4, an input to be mutated is selected from the seed corpus S . The **SelectSeed** function prioritizes inputs from the seed corpus that are likely to discover new paths. The selected input s is evaluated based on past execution results, and higher priority is given to inputs that are more likely to maximize code coverage. This approach focuses on exploring new code paths and uncovering potential bugs. In line 5, the selected input s is modified and altered through mutation methods to create a new input s' . The mutation process **Mutate** applies various techniques (bitflip, byteflip, splice, havoc, etc.) to modify and alter different parts of the input data. This is used to test how the program behaves with unexpected inputs. The mutated input(s') increases the likelihood of exploring new program paths and helps in effectively discovering potential vulnerabilities. In line 6, the mutated input is executed on the target program **Run**, and the result $result$ are returned. From lines 7 to 11, if the execution covers a new path **HasNewCoverage**, the mutated input is added to the seed corpus, and if it detect a new crash **DetectCrash**, it is added to

Algorithm 1: Fuzzing Algorithm

Input: pgm : a target program

S_{init} : a set of seed inputs

Output: $Cov(S)$: path coverage of seed inputs

$Crash$: crashing input corpus

```
1 Function FuzzTest( $pgm, S_{init}$ ):  
2    $S, Crash \leftarrow S_{init}, \emptyset$   
3   while not timeout do  
4      $s \leftarrow \text{SelectSeed}(S)$   
5      $s' \leftarrow \text{Mutate}(s)$   
6      $result \leftarrow \text{Run}(pgm, s')$   
7     if HasNewCoverage( $result$ ) then  
8        $S \leftarrow S \cup s'$   
9     else if DetectCrash( $result$ ) then  
10       $Crash \leftarrow Crash \cup s'$   
11    end  
12  end  
13  return  $S, Cov(S), Crash$   
14 End Function
```

the crashing input corpus $Crash$. If neither of these case occurs, the input is discarded. By repeating this process, more parts of the program are explored, and potential bugs and vulnerabilities can be discovered.

2.2. Multi-Armed Bandit(MAB) Problem

The Multi-Armed Bandit (MAB) problem (Sutton and Barto, 2018; Thompson, 1933) involves deciding which slot machine to choose among several to maximize the total reward obtained. Each slot machine has a different reward distribution, and the goal is to maximize the total reward obtained by repeatedly choosing among these slot machines within a limited time.

The solution to the Multi-Armed Bandit (MAB) problem typically relies on balancing the important reinforcement learning concepts of exploration and exploitation. Exploration involves trying out new slot machines to gather information about their rewards, while exploitation focuses on choosing the slot machine expected to yield the highest reward based on the information already known. In this process, decisions are made by considering the information available so far about the rewards of each slot machine and the uncertainty derived from that information. In other words, balancing exploration and exploitation hinges on reflecting the degree of uncertainty about how good the rewards from each slot machine will be.

Thompson sampling (Thompson, 1933; Chapelle and Li, 2011; Agrawal and Goyal, 2012) is one of the probabilistic algorithms used to solve this MAB problem by probabilistic modeling the reward distributions of each slot machine and

making decisions based on uncertainty. This algorithm expresses the uncertainty in rewards using the Beta distribution (Russo et al., 2020) and selects slot machines dynamically by sampling from this distribution, naturally balancing exploration and exploitation. By doing so, it enables more efficient decisions based on the success probability of each slot machine. The probability density function of the Beta distribution is given as follows:

$$f(x; \alpha, \beta) = \frac{\Gamma(\alpha + \beta)}{\Gamma(\alpha)\Gamma(\beta)} x^{\alpha-1} (1-x)^{\beta-1} \quad (1)$$

$$\Gamma(n) = (n-1)! \quad (2)$$

Equation (1) represents the probability density function of the Beta distribution. The parameters α and β are the two parameters of the Beta distribution, representing the number of successes and failures, respectively. Here, Γ denotes the Gamma function. We show in (2) the formula for the Gamma function. The gamma function (Artin, 2015; Sebah and Gourdon, 2002) is defined for all complex numbers except the non-positive integers. n is positive integer. The variable x in the equation represents the probability of success and ranges between 0 and 1. Thus, the Beta distribution is well-suited for expressing the uncertainty of the success probability and models the reward distribution for each slot machine by updating the values of α and β .

The Thompson sampling algorithm follows these steps:

1. *Initialization*: For each slot machine i , initialize α_i and β_i , setting both α_i and β_i to 1 at the start.
2. *Sampling*: For each slot machine i , sample a probability value from the current beta distribution $Beta(\alpha_i, \beta_i)$.
3. *Selection*: Choose the slot machine with the highest sampled probability value.
4. *Reward Observation and Update*: Update the α_i and β_i values for the chosen slot machine based on the reward outcome. If a reward is received (e.g., success), increase α_i by 1; if no reward is received (e.g., failure), increase β_i by 1.
5. *Repeat*: Repeat the above steps for the entire duration.

In this way, Thompson sampling probabilistically favors slot machines with higher estimated rewards while still allowing for the selection of slot machines with lower rewards based on values sampled from the probability distribution. This is because even slot machines with lower rewards or those not yet sufficiently evaluated can yield high values during the sampling process. By probabilistically selecting from all slot machines in this way, it naturally maintains a balance between exploration and exploitation. This algorithm is effectively used in fields such as online advertising campaign optimization and has been extensively researched and applied as a solution to the MAB problem.

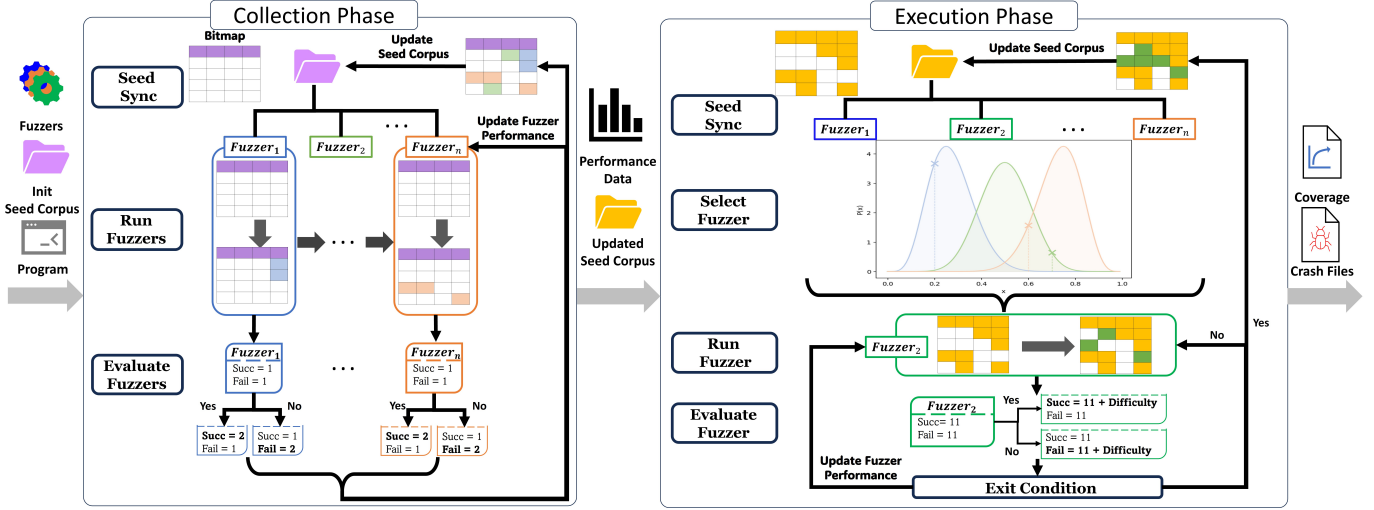


Figure 1: Overall process of RCFuzzer

We have formulated the fuzzer selection problem as a Multi-Armed Bandit (MAB) problem and applied Thompson sampling to solve it. In this formulation, each fuzzer is represented as an arm of the bandit, and its achieved code coverage serves as the corresponding reward. Through empirical analysis, we have demonstrated that the uncertainty in rewards for a fuzzer mostly follows a Beta distribution, aligning with the assumption of Thompson sampling (see Section 5.1).

3. APPROACH

3.1. Overall Process

Figure 1 shows the overall process of RCFuzzer. RCFuzzer takes the following items as inputs:

- A target program
- An initial seed corpus
- A set of individual fuzzers to orchestrate

RCFuzzer consists of the following two phases:

1. *Collection phase* (Section 3.2): RCFuzzer runs the fuzzing campaigns for each of the individual fuzzers in a given amount of time with the seed corpus and evaluates the performance of the individual fuzzers iteratively. The performance of each fuzzer is measured by counting the number of successful and failed campaigns. The fuzzing campaign is evaluated as successful if it improves code coverage or detects a crash, and failed otherwise.

Algorithm 2: Overall process of RCFuzzer

Input: pgm : a target program
 $\mathbb{F}$: a set of per-fuzzer data
 $\mathbb{S}_{init}$: an initial seed corpus
Output: $\mathbb{S}$: An updated seed corpus
 $Crash$: a set of detected crashes
 $Cov(\mathbb{S})$: code coverage achieved with $\mathbb{S}$

```
1 Function RCFuzzer ( $pgm, \mathbb{F}, \mathbb{S}_{init}$ ):  
2    $\mathbb{S} \leftarrow \mathbb{S}_{init}$   
3   foreach  $fz \in \mathbb{F}$  do  
4      $fz.S, fz.F \leftarrow 1, 1$   
5      $fz.diff, fz.threshold \leftarrow k_{diff}, k_{threshold}$   
6   end  
7    $\mathbb{F}, \mathbb{S}, Crash \leftarrow \text{Collect}(pgm, \mathbb{F}, \mathbb{S})$   
8   while Timeout is not reached do  
9      $\mathbb{F}, \mathbb{S}, Crash \leftarrow \text{Execute}(pgm, \mathbb{F}, \mathbb{S}, Crash)$   
10  end  
11  return  $\mathbb{S}, Crash, Cov(\mathbb{S})$   
12 End Function
```

After the fuzzing campaigns of the individual fuzzers finish, RCFuzzer updates the seed corpus with newly generated seed inputs and continues the loop until it reaches the timeout of the collection.

2. *Execution phase* (Section 3.3): RCFuzzer receives the performance evaluation results and updated seed corpus from the collection phase and iterates the collaborative fuzzing loop until it reaches the timeout of the fuzzing campaign. RCFuzzer uses Thompson sampling (see Section 2.2) to select the optimal fuzzer and update the evaluation result of the fuzzer. RCFuzzer first selects the optimal fuzzer for the current state of the target program using the current evaluation results of the fuzzers and runs the selected fuzzer in the timeout of fuzzing execution. Then, RCFuzzer evaluates the fuzzing execution as like RCFuzzer does in the collection phase and updates the number of success and fail counts for the fuzzer.

Algorithm 2 show how RCFuzzer works. In line 1, the seed corpus($\mathbb{S}$) is initialized as the initial seed corpus ($\mathbb{S}_{init}$). From lines 3 to 6, RCFuzzer initializes per-fuzzer data. In line 4, the success ($fz.S$) and fail ($fz.F$) counter for each fuzzer is initialized as 1 and 1, respectively. In line 5, the branch difficulty ($fz.diff$) and threshold ($fz.threshold$) values of each fuzzer is initialized as 100 (k_{diff}) and 10 ($k_{threshold}$).

In line 7, the **Collect** function, which corresponds to the collection phase, collects the performance of each fuzzer and returns the updated per-fuzzer data and code coverage as well as the detected crashes. After the collection phase finished, RCFuzzer goes to the execution phase. From lines 8 to 10, RCFuzzer

repeatedly calls **Execute** function which corresponds to the execution phase. **Execute** selects and executes the fuzzer expected to cover highest code coverage, and returns the updated per-fuzzer data through Thompson sampling. The execution phase is repeated until the timeout for overall RCFuzzer execution is reached.

3.2. Collection Phase

In the collection phase, RCFuzzer collects the per-fuzzer performance data by running fuzzing campaigns for each individual fuzzer and evaluates the performance of each fuzzer using the collected per-fuzzer performance data. The performance measured in the collection phase will be used as an input in the execution phase to select the optimal fuzzer in early iteration. Instead of applying Thompson sampling directly at the beginning, RCFuzzer separately evaluates the initial performance of each fuzzer in the collection phase. If Thompson sampling is applied from the very beginning, the fuzzer that is randomly selected initially may receive an unjustifiably high evaluation, which can distort fuzzing performance measurements and the accuracy of selecting the optimal fuzzer. This is because at the start of fuzzing, none of the code has been executed, making it easy to increase code coverage. As a result, a poorly performing fuzzer may be falsely regarded as a good fuzzer and given more opportunities, ultimately lowering the overall fuzzing efficiency. Although Thompson sampling will eventually assess fuzzer performance more accurately and reduce the likelihood of selecting underperforming fuzzers, the initial incorrect choices may lead to unnecessary time waste.

The algorithm presented in Algorithm 3 outlines the collection phase of RCFuzzer. It takes a target program pgm , a set of per-fuzzer data $\mathbb{F}$, and a seed corpus $\mathbb{S}$ as inputs and returns the updated set of per-fuzzer data $\mathbb{F}'$, an updated seed corpus $\mathbb{S}'$, and a set of detected crashes Crash' . In line 2, the collection phase begins by initializing the updated set $\mathbb{F}'$, $\mathbb{S}'$, and Crash' with the initial values of $\mathbb{F}$, $\mathbb{S}$, and an empty crash set $\emptyset$.

From lines 4 to 19, the collection phase runs for duration T which is initially assigned the value of the collection phase timeout parameter, τ_{collect} in line 3. For each iteration within the while loop (lines 4 to 19), the remaining time T is decremented by the minimum of 30 or the remaining time, T_{run} in line 5. During each iteration, the algorithm selects each fuzzer fz from the set $\mathbb{F}'$ and runs it for the duration of T_{run} using the function **RunFuzzer** in line 7. The algorithm monitors the progress of each fuzzer by comparing the difference in code coverage Cov between the current and previous states and whether any crashes have been detected in line 8.

If the fuzzer improves code coverage more than the fuzzer’s threshold $fz.threshold$ or finds a new crash, it increments the success count $fz.S$ and doubles the fuzzer’s threshold for future runs in lines 9–10. If no significant improvement is made, the failure count $fz.F$ is incremented and the fuzzer’s threshold is halved in lines 12–13. The updated seed corpus and crash set are expanded with each fuzzer’s individual results in lines 15–16. Finally, the collection phase terminates

Algorithm 3: Collection phase

Input: pgm : a target program
 $\mathbb{F}$: a set of per-fuzzer data
 $\mathbb{S}$: a seed corpus
Output: $\mathbb{F}'$: an updated set of per-fuzzer data
 $\mathbb{S}'$: an updated seed corpus
 $Crash'$: a set of crashes detected

```
1 Function Collect( $pgm, \mathbb{F}, \mathbb{S}$ ):  
2    $\mathbb{F}', \mathbb{S}', Crash' \leftarrow \mathbb{F}, \mathbb{S}, \emptyset$   
3    $T \leftarrow \tau_{collect}$   
4   while  $T > 0$  do  
5      $T_{run}, \mathbb{S}_{prev}, Crash_{prev} \leftarrow \min(T, 30), \mathbb{S}', Crash'$   
6     foreach  $fz \in \mathbb{F}'$  do  
7        $\mathbb{S}_{fz}, Crash_{fz} \leftarrow \text{RunFuzzer}(pgm, fz, \mathbb{S}_{prev}, T_{run})$   
8       if  $|Cov(\mathbb{S}_{fz}) - Cov(\mathbb{S}_{prev})| > fz.threshold$  or  
9          $|Crash_{fz} - Crash_{prev}| \neq 0$  then  
10         $fz.S \leftarrow fz.S + 1$   
11         $fz.threshold \leftarrow fz.threshold * 2$   
12      else  
13         $fz.F \leftarrow fz.F + 1$   
14         $fz.threshold \leftarrow fz.threshold * 0.5$   
15      end  
16       $\mathbb{S}' \leftarrow \mathbb{S}' \cup \mathbb{S}_{fz}$   
17       $Crash' \leftarrow Crash' \cup Crash_{fz}$   
18    end  
19     $T \leftarrow T - T_{run}$   
20  end  
21  return  $\mathbb{F}', \mathbb{S}', Crash'$   
22 End Function
```

when the time T runs out, and the function returns the updated per-fuzzer data $\mathbb{F}'$, the updated seed corpus $\mathbb{S}'$, and the new set of crashes $Crash'$.

This phase is essential for gathering performance data on each fuzzer and dynamically adjusting their behavior based on coverage and crash detection. The algorithm employs a per-fuzzer threshold $fz.threshold$ to dynamically adjust the evaluation criteria for each fuzzer based on its individual performance during the collection phase. This approach enables the algorithm to adaptively manage the contribution of each fuzzer by modifying its success and fail evaluation criteria over time. When a fuzzer successfully expands code coverage or detects a new crash, its threshold is increased by double, requiring it to achieve more significant improvements in future iterations. On the other hand, if a fuzzer fails to make progress, the threshold is reduced by half, allowing the fuzzer more chances to contribute even with smaller improvements. This mechanism helps to balance exploration and exploitation by giving well-performing

Algorithm 4: Execution phase

Input: pgm : a target program $\mathbb{F}$: a set of per-fuzzer data $\mathbb{S}$: a seed corpus $Crash$: a set of crashes previously detected**Output:** $\mathbb{F}'$: an updated set of per-fuzzer data $\mathbb{S}'$: an updated seed corpus $Crash'$: an updated set of crashes with newly detected

```
1 Function Execute( $pgm, \mathbb{F}, \mathbb{S}, Crash$ ):
2    $\mathbb{F}', \mathbb{S}', Crash' \leftarrow \mathbb{F}, \mathbb{S}, Crash$ 
3    $T, numFailed \leftarrow \tau_{exe}, 0$ 
4    $fz \leftarrow \text{SelectFuzzer}(\mathbb{F}')$ 
5   while  $T > 0$  or  $numFailed < 5$  do
6      $T_{run}, \mathbb{S}_{prev}, Crash_{prev} \leftarrow \min(T, 60), \mathbb{S}', Crash$ 
7      $\mathbb{S}_{fz}, Crash_{fz} \leftarrow \text{RunFuzzer}(pgm, fz, \mathbb{S}_{prev}, T_{run})$ 
8     if  $|Cov(\mathbb{S}_{fz}) - Cov(\mathbb{S}_{prev})| > fz.threshold$  or
        $|Crash_{fz} - Crash_{prev}| \neq 0$  then
9        $numFailed \leftarrow 0$ 
10       $fz.threshold \leftarrow fz.threshold * 2$ 
11       $\text{UpdateFuzzer}(fz, 'success')$ 
12    else
13       $numFailed \leftarrow numFailed + 1$ 
14       $fz.threshold \leftarrow fz.threshold * 0.5$ 
15       $\text{UpdateFuzzer}(fz, 'failure')$ 
16    end
17     $\mathbb{S}' \leftarrow \mathbb{S}' \cup \mathbb{S}_{fz}$ 
18     $Crash' \leftarrow Crash' \cup Crash_{fz}$ 
19     $T \leftarrow T - T_{run}$ 
20  end
21  return  $\mathbb{F}', \mathbb{S}', Crash'$ 
22 End Function
```

fuzzers more demanding criteria while giving less productive fuzzers more opportunities before deprioritizing them. By doing so, the algorithm ensures efficient resource allocation and avoids prematurely discarding potentially useful fuzzers, ultimately enhancing the overall fuzzing effectiveness.

3.3. Execution Phase

The goal of the execution phase is to select the most efficient fuzzer based on the performance data collected in each round, in order to generate test cases that satisfy increasingly complex path conditions. To achieve this, two approaches are employed: recommendation-based selection(Section 3.3.1) and a branch difficulty-based heuristic(Section 3.3.2).

The algorithm presented in Algorithm 4 outlines the execution phase of the fuzzing process. The function `Execute` takes a target program `pgm`, a set of per-fuzzer data $\mathbb{F}$, a seed corpus $\mathbb{S}$, and a set of previously detected crashes *Crash* as inputs. The goal is to update the set of per-fuzzer data, seed corpus, and the list of detected crashes based on fuzzing performance during the execution phase.

The algorithm begins by initializing the updated sets $\mathbb{F}'$, $\mathbb{S}'$, and *Crash'* with the initial values of $\mathbb{F}$, $\mathbb{S}$, and *Crash* in line 2. It sets the total execution time T to a predefined timeout τ_{exe} , and initializes *numFailed* to track the number of consecutive failures in line 3. A fuzzer *fz* is selected from the set $\mathbb{F}'$ using the `SelectFuzzer` function in line 4. The fuzzing loop continues while the time T is greater than 0 or the number of consecutive failures *numFailed* is less than 5 in line 5. If there are consecutive failures, RCFuzzer terminates the fuzzing loop early; otherwise, the fuzzing loop continues until the execution phase timeout is reached. The execution phase proceeds with updated evaluation results until the overall timeout of the fuzzing campaign is reached.

Within each iteration, the algorithm sets the runtime for the fuzzer T_{run} to the minimum of T and 60 seconds, and captures the previous seed corpus $\mathbb{S}_{prev}$ and crash set *Crash_{prev}* in line 6. The selected fuzzer *fz* is then run on the target program using `RunFuzzer`, resulting in an updated seed corpus $\mathbb{S}_{fz}$ and crash set *Crash_{fz}*. If the coverage improves beyond the fuzzer’s threshold *fz.threshold* or a new crash is detected, the failure count *numFailed* is reset to 0, and the threshold is doubled to make future evaluations stricter in lines 9–10.

The fuzzer is updated with a success status in line 11. Otherwise, *numFailed* is incremented, and the threshold is halved to allow more lenient evaluation in future iterations in lines 13–14. The fuzzer is updated with a failure status in line 15.

The updated seed corpus and crash set are combined with $\mathbb{S}'$ and *Crash'* respectively, and the remaining time T is reduced by T_{run} . This process continues until the loop conditions are no longer satisfied, after which the function returns the updated sets $\mathbb{F}'$, $\mathbb{S}'$, and *Crash'*. This phase ensures that each fuzzer’s performance is dynamically adjusted and the overall fuzzing process is refined based on continuous feedback.

3.3.1. Recommendation-based Selection

RCFuzzer utilizes Thompson sampling (Thompson, 1933; Chapelle and Li, 2011; Agrawal and Goyal, 2012) to analyze the performance of each fuzzer and dynamically select the fuzzer expected to improve code coverage and detect a bug, thereby maximizing code coverage and crash detection capability. This approach effectively handles the uncertainty arising from variations in the performance of each fuzzer due to various factors such as the target program, input data, and execution conditions. RCFuzzer continuously updates the success probabilities of each fuzzer based on observed successes and fails, and dynamically selects fuzzers according to these probabilities to adapt to performance changes.

Algorithm 5: Thompson Sampling

Input : *fuzzers* : a set of per-fuzzer data

Output: *selectedFuzzer*: the chosen fuzzer

1 **Function** *SelectFuzzer*(*fuzzers*):

2 **foreach** *fz* $\in$ *fuzzers* **do**

3 | Sample $\theta_{fz} \sim \text{Beta}(fz.S, fz.F)$

4 **end**

5 *selectedFuzzer* $\leftarrow \text{argmax}_{fz}(\theta_{fz})$

6 **return** *selectedFuzzer*

7 **End Function**

Input : *fuzzers*: the per-fuzzer data to be updated

result: the execution result of the fuzzer

8 **Function** *UpdateFuzzer*(*fuzzer*, *result*):

9 **if** *result* is success **then**

10 | *fuzzer.S* $\leftarrow$ *fuzzer.S* + *fuzzer.diff*

11 | *fuzzer.diff* $\leftarrow \lfloor \text{fuzzer.diff} * 0.5 \rfloor$

12 **else**

13 | *fuzzer.F* $\leftarrow$ *fuzzer.F* + *fuzzer.diff*

14 | *fuzzer.diff* $\leftarrow$ *fuzzer.diff* + 1

15 **end**

16 **End Function**

In Thompson sampling, each fuzzer is modeled as a slot machine, where different machines yield different rewards, such as increased code coverage or the detection of new crashes. For each fuzzer, the parameters α and β of the beta distribution are adjusted based on the number of successes and failures, representing the probability of the fuzzer’s success. Probabilities are sampled from each fuzzer’s beta distribution, and the fuzzer with the highest sampled value is selected for the next fuzzing campaign (line 4 in Algorithm 4) in the execution phase. Fuzzers with a higher number of successes are more likely to be selected. However, fuzzers that have lower number of successes also have a chance to be chosen to maintain a balance between exploitation and exploration. This ensures that RCFuzzer primarily exploits fuzzers with high success probabilities while still exploring underutilized fuzzers, allowing for potential code coverage improvement and the crash detection. By maintaining this balance between exploration and exploitation, RCFuzzer dynamically selects the optimal fuzzer and maximizes the overall fuzzing effectiveness.

The algorithm in Algorithm 5 describes the process of selecting a fuzzer using Thompson sampling, followed by updating the fuzzer’s success and failure parameters based on its performance. **SelectFuzzer**, takes as input a set of per-fuzz data, each of which has success (*fz.S*) and failure (*fz.F*) counts. For each fuzzer, the algorithm samples a value θ_f from the fuzzer’s beta distribution,

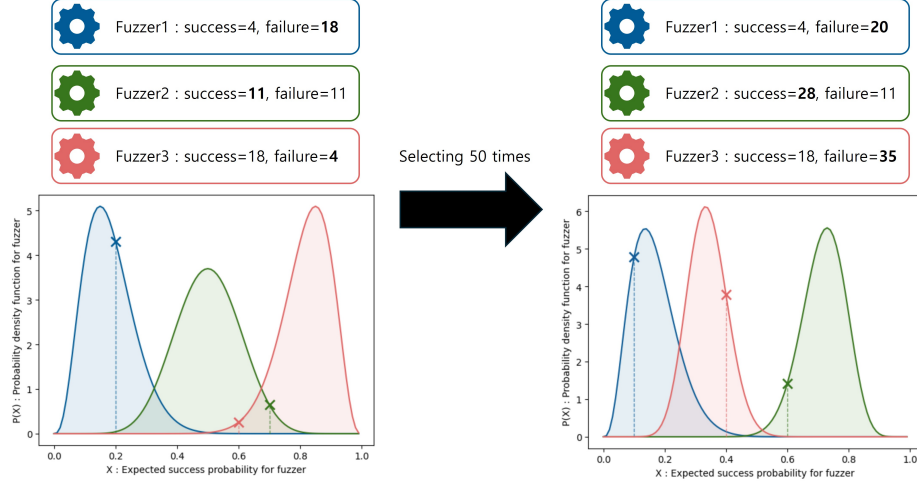


Figure 2: Example of the recommendation-based fuzzer selection

where the beta distribution is parameterized by its success and fail counts. The fuzzer with the highest sampled value is chosen as the *selectedFuzzer* using the *argmax* function. This process allows the algorithm to balance exploration of underperforming fuzzers with the exploitation of well-performing ones. The second function, *UpdateFuzzer* implements the branch difficulty-based heuristic (Section 3.3.2).

Figure 2 illustrates an example of recommendation-based fuzzer selection using Thompson sampling. The graph depicts the beta distribution for each fuzzer, where $P(X)$ represents the probability density function (PDF) of the expected success probability (X) for each fuzzer.

In the first graph, Fuzzer1 has 4 successes and 18 failures, Fuzzer2 has 11 successes and 11 failures, and Fuzzer3 has 18 successes and 4 failures. The values sampled from the beta distribution for each fuzzer are 0.2, 0.7, and 0.6, respectively, leading to the selection of Fuzzer2 as it has the highest sampled value. These sampled values reflect the current estimated success probabilities of the fuzzers and are dynamically updated in each round. As a result, fuzzers with higher success probabilities are selected more frequently.

The second graph demonstrates the changes in the beta distribution for each fuzzer after 50 additional selections. In this example, we assume the number of successes and failures increases incrementally. Fuzzer2 was selected 14 times and succeeded in each case, causing its distribution to shift rightward, indicating an increased expected success probability. Meanwhile, Fuzzer3 was selected 31 times but failed each time, resulting in its distribution shifting toward the center, illustrating a decrease in expected success probability. Fuzzer1, selected only twice and failing both times, experiences a slight leftward shift in its distribution. As fuzzers accumulate successes, their beta distribution moves rightward and concentrates around higher success probabilities. Conversely, failures cause the

distribution to shift leftward and center around lower probabilities.

As shown in Figure 2, this recommendation-based fuzzer selection strategy dynamically adjusts selection probabilities based on fuzzer performance. A successful fuzzer gradually accumulates a higher expected success probability, increasing its chances of being selected, while a fuzzer with frequent failures becomes less likely to be chosen. Importantly, the fuzzer with the highest number of successes is not always selected; instead, selection is based on the value sampled from each fuzzer’s beta distribution, making the process more flexible and efficient. This approach enhances fuzzing efficiency by frequently selecting the most effective fuzzers while still allowing for exploration of others.

3.3.2. The Branch Difficulty-based Heuristic

The branch difficulty-based heuristic approach, combined with the AIMD (Additive Increase, Multiplicative Decrease) method (Chiu and Jain, 1989), enhances fuzzing performance by maintaining a balance between exploration and exploitation, even under complex path conditions, allowing for the selection of the optimal fuzzer. This approach plays a crucial role in selecting optimal fuzzers as path conditions become more complex, helping to reduce waste of time.

As the difficulty of executing uncovered code grows increasingly over time during the fuzzing process, it becomes more challenging to explore new paths. Consequently, the selected fuzzer experiences an increase in failures and a decrease in successes. This leads to the underestimation of the performance of the fuzzer that spend a significant amount of time to cover the difficult paths, resulting in the selection of less effective fuzzers and wasting time. To address this issue, we propose the branch difficulty-based fuzzer weighting method.

By assigning higher weights to fuzzers that solve complex path conditions and increasing their success counts, we can mitigate some of these problems. However, this approach may cause the success counts of fuzzers that solve complex paths to rapidly increase, leading to their continuous selection and potentially disrupting the balance between exploration and exploitation. This could result in an overall decline in fuzzing performance.

To prevent this, we introduced the AIMD method. AIMD halves the branch difficulty upon success, providing opportunities for exploration, and increases the branch difficulty by one upon failure, encouraging the fuzzer to tackle more complex paths. This method helps maintain the balance between exploration and exploitation, improving the efficiency of fuzzers. The AIMD formula is as follows:

$$diff_{t+1} = \begin{cases} diff_t \times 0.5, & \text{if success} \\ diff_t + 1, & \text{if failure} \end{cases} \quad (3)$$

UpdateFuzzer in Algorithm 5, adjusts the success ($fuzzer.S$) and failure ($fuzzer.F$) parameters of the chosen fuzzer based on the result of its execution. If the result is a success, the fuzzer’s success count ($fuzzer.S$) is incremented by the branch difficulty $fuzzer.diff$, which is a dynamic adjustment value that

Table 1: Changes in the success and failure counts of each fuzzer

Round	Fuzzer A		Fuzzer B		Fuzzer C	
	#success	#failure	#success	#failure	#success	#failure
Init	1	1	1	1	1	1
1	2	1	2	1	2	1
2	3	1	3	1	2	2
$\vdots$	$\vdots$					
20	4	18	11	11	18	4

halves with each success, ensuring smaller adjustments over time. Conversely, if the result is a failure, the failure count ($fuzzer.F$) is incremented by the branch difficulty $fuzzer.diff$, which grows by 1 with each failure. This dynamic adjustment mechanism helps fine-tune the learning process for each fuzzer, allowing for a more accurate assessment of its effectiveness in the long run.

In conclusion, the combination of the branch difficulty-based heuristic approach and the AIMD method ensures the selection of the most effective fuzzer by balancing exploration and exploitation, even under complex path conditions. This approach contributes to maximizing overall fuzzing performance by continuously exploring new paths and solving high-difficulty ones.

3.4. An Illustrative Example of RCFuzzer

To better understand the execution of RCFuzzer, consider an example with three individual fuzzers: A, B, and C. The input consists of a target program, an initial seed corpus ($\mathbb{S}_{init}$), and the three fuzzers.

3.4.1. Collection Phase

During the collection phase, RCFuzzer runs each fuzzer independently for the same duration using the initial seed corpus. At the beginning, the success and failure counts of all fuzzers are initialized to 1. Table 1 shows the changes in the success and failure counts of each fuzzer in each round of the collection phase.

The collection phase follows these steps:

- In the first round, all fuzzers generate test cases based on the initial seed corpus and increase code coverage. As a result, each fuzzer is marked successful, and their success counts increase from 1 to 2 (see the third row in Table 1).
- In the second round, Fuzzer A improves coverage, Fuzzer B detects a new crash, and Fuzzer C fails to make any progress. Consequently, A and B’s success counts increase by 1, while C’s failure count increases by 1 (see the fourth row in Table 1).

- This process repeats until the collection phase timeout (e.g., $\tau_{collect} = 600$) is reached. In this example, 20 rounds were executed and the number of success and failure for each fuzzer are recorded. (see the last row in Table 1).

After the collection phase finishes, RCFuzzer knows the initial performance of each individual fuzzers, which is used in the execution phase to select the optimal fuzzer.

3.4.2. Execution Phase

After the collection phase, RCFuzzer utilizes Thompson sampling to select the optimal fuzzer based on the collected performance data. The selection process follows these steps:

1. Each fuzzer’s performance is modeled using a Beta distribution, parameterized by its recorded success and failure counts.
2. A probability value is sampled from each fuzzer’s Beta distribution.
3. The fuzzer with the highest sampled value is selected for execution.

For example, suppose the final success and failure counts from the collection phase are:

- Fuzzer A: 4 successes, 18 failures
- Fuzzer B: 11 successes, 11 failures
- Fuzzer C: 18 successes, 4 failures

If the sampled values from the Beta distributions are 0.2 for A, 0.7 for B, and 0.6 for C, then Fuzzer B is selected for execution. The left graph in Figure 3 visualizes this process by showing the distribution of success and failure counts along with the corresponding sampled values, which are represented by X markers.

Once a fuzzer is selected and executed, its performance is reassessed:

- If the fuzzer improves code coverage or detects a new crash, its success count is increased by the branch difficulty value (k_{diff}), and the difficulty value is halved for the next iteration.
- If the fuzzer fails, its failure count is increased by k_{diff} , and k_{diff} is incremented by 1.

Suppose that the initial value of the branch difficulty is 100. If Fuzzer B’s execution is successful, its success count (=11) increases by the branch difficulty value (=100) and the branch difficulty value is halved to 50 because the current difficulty of covering a new branch is relatively easy. If unsuccessful, fuzzer B’s failure count (=11) increases by the branch difficulty value (=100), and the branch difficulty value increases by 1.

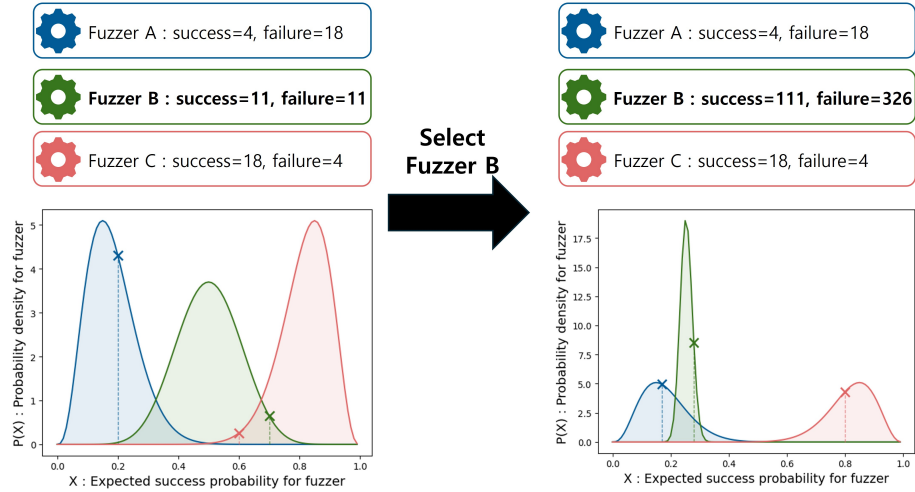


Figure 3: Example of the execution phase

This execution process continues until the timeout for the execution phase is reached. If the selected fuzzer continuously fails, an early termination condition is triggered, and the next fuzzer is chosen. For instance, if fuzzer B succeeds once but then fails five times in a row, its updated counts would be: Fuzzer B with 111 successes ($= 11 + 100$) and 326 failures ($= 11 + 50 + 51 + 52 + 53 + 54 + 55$). Since fuzzer B exceeds the allowed consecutive failure limit, early termination occurs, and a new fuzzer selection process begins based on the latest success and failure counts. The second graph in Figure 3 visualizes the changes in the success and failure counts after this process, illustrating how the fuzzer's performance updates over time.

By iterating through these steps, RCFuzzer dynamically selects the most effective fuzzer, maximizing code coverage and bug detection efficiency while maintaining a balance between exploration and exploitation.

4. EXPERIMENT SETUP

4.1. Research Questions

We set the following five research questions to evaluate the performance of RCFuzzer.

- **RQ1.** How much effective is RCFuzzer compared with the individual fuzzers?
- **RQ2.** How much effective is RCFuzzer compared with the state-of-the-art collaborative fuzzers?
- **RQ3.** Does the collection-execution strategy improve the effectiveness of RCFuzzer?

- **RQ4.** Does the recommendation-based fuzzer selection affect the performance of RCFuzzer?
- **RQ5.** How much does the branch difficulty-based heuristic affect the performance of RCFuzzer?

4.2. RCFuzzer Setup

4.2.1. Individual Fuzzers Used

Table 2: Individual fuzzer

Fuzzer	Description
AFL (Zalewski, 2017)	It utilizes coverage-guided fuzzing to maximize code coverage by analyzing running program data.
FairFuzz (Lemieux and Sen, 2018)	It focuses on uncovering new vulnerabilities by fuzzing rarely explored paths.
AFLFast (Böhme et al., 2016)	It is designed to discover vulnerabilities faster by optimizing the priority of execution paths through coverage feedback.
MOpt (Lyu et al., 2019)	It automatically optimizes mutation strategies using genetic algorithms, selecting the best mutation strategy to enhance coverage.
LearnAFL (Yue et al., 2019)	It enhances fuzzing efficiency by using machine learning to learn from code coverage data and automatically generate effective mutation strategies.
LAF-INTEL (Intel, 2016)	It expands coverage information by refining conditionals, enabling more granular exploration of the code.
RedQueen (Aschermann et al., 2019)	It analyzes the state of the running program by tracking memory states, generating meaningful mutations to explore new execution paths.
Radamsa (Helin, 2018)	It is a black-box fuzzer that mutates input data randomly without using execution path information or coverage feedback.
Angora (Chen and Chen, 2018)	It utilizes type information of inputs to perform mutations, ensuring the inputs meet specific conditions.
QSYM (Yun et al., 2018)	It is a hybrid fuzzing tool that combines AFL with symbolic execution to enhance coverage.
libFuzzer (Project, 2015)	It performs in-process, coverage-guided fuzzing by directly calling functions with mutated inputs and analyzing code coverage.

In the experiments, RCFuzzer utilized 11 individual fuzzers. Table 2 describes a brief description of the 11 individual fuzzers. We selected these 11 fuzzers because they have their own unique characteristics which can make a

synergistic effect. In addition, these fuzzers are employed by the state-of-the-art collaborative fuzzer autofz (Fu et al., 2023), which allows for a fair comparison between RCFuzzer and autofz.

4.2.2. Parameter Configuration

We set the execution time of the single execution of both the collection phase and the execution phase to 600 seconds ($\tau_{collect} = 600$ and $\tau_{exe} = 600$). We initially set the values of branch difficulty and threshold to 100 and 10, respectively ($k_{diff} = 100$ and $k_{threshold}=10$). To find reasonable values for these parameters, we performed an exploratory study with various sets of parameter values.

Table 3 shows the results of an exploratory study conducted on five target programs with various parameter combinations. A total of 81 parameter value combinations were evaluated by varying the collection time ($\tau_{collect}$) and execution time (τ_{exe}) between 300, 600, and 900 seconds, the branch difficulty (k_{diff}) between 50, 100, and 200, and the threshold ($k_{threshold}$) between 5, 10, and 20. We selected five programs from different domains. We selected `boringssl`, `libarchive`, and `libxml2` from the Fuzzer-Test-Suite (FTS) benchmark (Google, 2021), and `pdftotext` and `tcpdump` from the UNIFUZZ benchmark (Li et al., 2021). These programs vary in source code size, ranging from a minimum of 9,122 lines to a maximum of 133,762 lines.

Through the exploratory study, we found that while the optimal parameter combination varied across individual programs, the combination of parameter (No.28 - $\tau_{collect}=600$, $\tau_{exe}=600$, $k_{diff}=100$, $k_{threshold}=10$) achieved the highest line coverage, on average. Additionally, we analyzed nine combinations that fix the $\tau_{collect}$ and τ_{exe} at 600 seconds while varies only the k_{diff} and $k_{threshold}$. As a result, the combination($k_{diff}=100$, $k_{threshold}=10$) achieved the best performance in terms of code coverage among the nine combinations.

4.2.3. Implementation

We have implemented RCFuzzer on top of autofz. The core components of RCFuzzer are implemented in 1,000 lines of Python code. To implement Thompson sampling, we have used the `numpy` package. The implementation of RCFuzzer is publicly available at <https://github.com/hyeonminmo/RCFuzzer>

4.3. Baseline Fuzzers

For RQ1, we have used the 11 individual fuzzers RCFuzzer used as the individual fuzzers of the baseline. For RQ2, we used autofz as the collaborative fuzzer of baseline. ENFUZZ (Chen et al., 2019) and CUPID (Güler et al., 2020) were not included in the experiment Fu et al. (Fu et al., 2023) have shown that autofz outperformed ENFUZZ and CUPID. RQ3, RQ4 and RQ5 were evaluated by comparing RCFuzzer with its variants. The following are the variants of RCFuzzer used in the experiments.

- **RCFuzzer^{Collect-}** : It is the same to RCFuzzer except that it does not have an collection phase.

Table 3: Evaluation of parameter combinations

[illegible]

Bold and Italic text indicates a parameter combination that achieved the best performance. **Bold** text indicates the item with the best performance for each program.

- **RCFuzzer** ^{*Rand*} : It is the same to RCFuzzer except that it selects a fuzzer randomly instead of using Thompson sampling.
- **RCFuzzer** ^{*Diff⁻*} : It is the same to RCFuzzer except that it does not have a branch difficulty-based heuristic method.

4.4. Benchmark

We used three widely recognized benchmarks: the MAGMA benchmark (Hazimeh et al., 2020), the UNIFUZZ benchmark (Li et al., 2021), and the Fuzzer-Test-Suite (FTS) (Google, 2021) benchmark. These benchmarks provide real-world programs from various software domains with seed inputs, allowing for a more comprehensive and fair evaluation of fuzzers’ performance. MAGMA, UNIFUZZ, and FTS are widely used in the fuzzing literature (Zhang et al., 2023; Li et al., 2024; Zhao et al., 2024; Hu et al., 2024; Yu et al., 2024; Zhang et al., 2024b,a; Wu et al., 2024; Kong et al., 2025).

Table 4 shows the target programs, their software domains, and size in terms of Lines of Code (LoC) for each benchmark. These benchmarks span 13 different software domains, including security, networking, and image processing, with program sizes ranging from 1,502 LoC (`jhead`) to 182,927 LoC (`php-unserialize`). We used a total of 47 programs, excluding those that could not be compiled. Among them, 24 were from FTS, 14 from UNIFUZZ, and 9 from MAGMA ¹.

FTS includes various versions of `OpenSSL`, such as 1.0.1f, 1.0.2f, `bignum`, and `x509`. Although `bignum` and `x509` both use version 1.1.0c, they are classified separately because `bignum` focuses on large number computations while `x509` is used for certificate-related tasks. Similarly, `OpenThread`’s `ip6` and `radio`, both based on the 2018-02-27 version, are classified separately as `ip6` pertains to the networking layer protocol, whereas `radio` operates at the physical and data link layers. In the MAGMA benchmark, the same software package can have multiple executables. For example, `libtiff` has two executables, `read_rgba` and `tiffcp` which have distinct functionalities. `read_rgba` converts pixel data to RGBA format, whereas `tiffcp` merges multiple TIFF files or performs specific transformations. In such cases, we treated each executable as a separate target program as we do for `OpenSSL` in FTS. Similarly, `PHP` in MAGMA have four different executables.

Note that `libFuzzer` could not be applied to the target programs in UNIFUZZ. This is because these programs do not provide the required fuzzing harness (i.e., `LLVMFuzzerTestOneInput`) needed for `libFuzzer`. Similarly, `Angora` was excluded from the experiments on MAGMA’s target programs due to un-

¹We excluded the target programs in MAGMA, which are also included in FTS and/or UNIFUZZ due to time constraints.

Table 4: Target programs, domains, and lines of code for each benchmark

Bench- mark	Program	Domain	Size (LoC)
Fuzzer-Test-Suite(FTS)	boringssl	Cryptography and Security	66,085
	c-ares	Networking	4,382
	guetzli	Image Processing	5,320
	harfbuzz	Font Management	10,938
	json	Text Processing	3,197
	lcms	Image Processing	16,806
	libarchive	Archive Management	46,851
	libjpeg-turbo	Image Processing	18,500
	libpng	Image Processing	10,165
	libssh	Cryptography and Security	19,589
	libxml2	Text Processing	133,762
	openssl-1.0.1f	Cryptography and Security	100,934
	openssl-1.0.2d	Cryptography and Security	114,038
	openssl-bignum	Cryptography and Security	116,897
	openssl-x509	Cryptography and Security	116,897
	openthread-ip6	Networking	30,575
	openthread-radio	Networking	30,575
	pcre2	Text Processing	12,072
	proj4	Geospatial Data Transformation	15,772
	re2	Text Processing	9,087
UNIFUZZ	sqlite	Database Management	38,633
	vorbis	Audio Processing	8,373
	woff2	Font Management	11,263
	wpantund	Networking	15,538
	cflow	Software Analysis and Debugging	5,706
	exiv2	Image Processing	27,969
	flvmeta	Audio Processing	6,997
	imginfo	Image Processing	14,443
	infotocap	System Utilities	32,487
	jhead	Image Processing	1,502
	lame	Audio Processing	11,292
	mp3gain	Audio Processing	3,497
	mp42aac	Audio Processing	23,744
MAGMA	nm	Software Analysis and Debugging	51,452
	pdftotext	Documents Processing	51,067
	sqlite3	Database Management	34,435
	tcpdump	Networking	9,122
	tiffsplit	Image Processing	24,082
	libsndfile	Audio Processing	22,311
	libtiff-read_rgba	Image Processing	15,969
	libtiff-tiffcp	Image Processing	15,056
	lua	Programming Language	10,342
	php-exif	Programming Language	182,923
	php-json	Programming Language	182,924
	php-parser	Programming Language	182,917
	php-unserialize	Programming Language	182,927
	poppler-pdf	Documents Processing	91,626

resolved technical issues that prevented its successful execution ². As a result,

²We have tried our best to fix the issues by contacting the authors of Angora and MAGMA,

experiments on UNIFUZZ and MAGMA were conducted excluding libFuzzer and Angora, respectively.

All the experiments used the default seed inputs provided by each benchmark.

4.5. Measurement

To evaluate the effectiveness of RCFuzzer and the other fuzzers, we measured the branch coverage, the number of unique crashes detected, and the memory usage.

4.5.1. Line Coverage

To measure the line coverage, we ran the target programs with all the generated test inputs and used `afl-cov` (Zalewski, 2017) to obtain the number of lines covered.

4.5.2. Unique Crash Detection

To measure the number of unique crashes detected, we collected all the crashes detected and remove duplicated crashes. First, we used AddressSanitizer (Serebryany et al., 2012; Google, a) and MemorySanitizer (Google, b) to collect crashes caused by the generated test inputs. Then, we analyzed the stack trace of the crashes to automatically remove the duplicated crashes. We compared the top three entries of the stack trace and classified crashes with identical stack traces as duplicated. Finally, we manually identified the unique crashes among the duplicated crashes that had different stack traces but were likely caused by the same root cause. In this process, we determined the root cause of the crashes based on the bug fix information such as the bug patch, the bug reports.

4.6. Fuzzing Campaign Setup

We ran the experiments on a machine equipped with two AMD EPYC 7763 processors with 64 cores and 512 GB memory running Ubuntu 22.04.3 LTS. Following the guideline on fuzzer evaluation proposed by Klees et al. (Klees et al., 2018), we conducted our evaluation accordingly. To address random variance, each fuzzer was executed in 10 independent trials. In each trial, we run RCFuzzer and the baseline fuzzers for 24 hours. We reported the average line coverage and all the detected unique crashes over the 10 runs as results.

4.7. Threats to Validity

External. External validity concerns whether the findings of the study can be generalized beyond the specific experiment settings. One threat to external validity is the representativeness of our target programs. To limit this threat, we used 47 programs from the three widely used benchmarks which covers 12

but we could not resolve the issues.

software domains. Another threat to external validity is the selection of the 11 individual fuzzers. We tried to reduce this threat by selecting the fuzzers that have been widely used and has different characteristics to represent various fuzzers.

Internal. Internal validity concerns whether the observed effects in the study were actually caused by the proposed method (RCFuzzer) rather than other factors. A threat to internal validity is possible faults in the RCFuzzer implementations. To address this threat, we thoroughly tested the implementation. Another threat to internal validity is the parameter configuration. To address this threat, we performed an exploratory study to find the optimal parameter values.

Construct. Construct validity assesses whether the study accurately measures what it claims to measure. A threat to construct validity is the use of code coverage and the number of unique crashes as performance indicators. We believe this threat is limited because they are standard fuzzing performance indicators used in the literature.

Conclusion. Conclusion validity concerns whether the conclusions drawn from the study are valid. One threat to conclusion validity is that our results may be influenced by the specific parameter settings used in RCFuzzer. Although we conducted an exploratory study over 81 combinations and selected the configuration with the best average performance across five diverse programs (see Section 4.2.2 and Table 3), this choice may still favor certain program characteristics. As different programs may benefit from different settings, the observed improvements could partially reflect favorable tuning rather than algorithmic superiority. Thus, conclusions about RCFuzzer’s effectiveness should be interpreted with this sensitivity in mind.

Another threat concerns statistical significance. We applied the Mann-Whitney U test (Mann and Whitney, 1947) to assess whether the performance differences between RCFuzzer and baseline techniques were significant. Statistically significant cases are marked in *italic* in the coverage tables in Section 5.2 through Section 5.6. Although not all improvements in line coverage were statistically significant, RCFuzzer consistently showed superior crash detection compared to the baseline fuzzers.

5. EXPERIMENT RESULTS

5.1. Empirical Validation of the Beta Distribution Assumption in Thompson Sampling for Fuzzer Selection

In this study, we verified whether the uncertainty of the reward of fuzzer follows a Beta distribution, which is the assumption of Thompson sampling. We used the Kolmogorov-Smirnov test (Massey Jr, 1951; Smirnov, 1948) to evaluate the goodness of fit between the observed data and the Beta distribution. The validity of the Beta distribution assumption was assessed based on the p -value. We used 0.05 as the significance level, and a p -value greater than 0.05 was considered to indicate a good fit with the Beta distribution (Fisher, 1970).

Table 5: p -value of the fuzzers for each target program

Program	AFL	FairFuzz	AFLFast	MOpt	LearnAFL	LAF-INTEL	RedQueen	Radamsa	Angora	QSYM	libFuzzer
boringsssl	0.943	0.442	0.569	0.899	0.994	0.979	0.985	0.897	0.639	0.990	0.716
c-ares	0.724	0.324	0.190	0.590	0.631	0.949	0.443	0.483	0.604	0.447	0.528
cflow	0.294	0.295	0.339	0.924	0.980	0.568	0.768	0.295	0.526	0.792	-
exiv2	0.234	0.839	0.432	0.511	0.408	0.398	0.314	0.660	0.986	0.354	-
flvmeta	0.439	0.735	0.997	0.558	0.873	0.982	0.039	0.729	0.730	0.543	-
guetzli	0.219	0.435	0.257	0.770	0.584	0.026	0.828	0.106	0.637	0.665	0.908
harfbuzz	0.087	0.489	0.052	0.726	0.758	0.059	0.160	0.160	0.249	0.259	0.700
imginfo	0.000	0.000	0.000	0.000	0.887	0.026	0.000	0.402	0.989	0.000	-
infotocap	0.030	0.068	0.469	0.753	0.495	0.075	0.081	0.694	0.956	0.670	-
jhead	0.000	0.653	0.555	0.713	0.596	0.004	0.000	0.000	0.488	0.580	-
json	0.462	0.726	0.810	0.323	0.753	0.970	0.788	0.400	0.672	0.821	0.903
lame	0.370	0.847	0.442	0.283	0.616	0.601	0.440	0.634	0.334	0.389	-
lcms	0.003	0.003	0.074	0.745	0.900	0.000	0.000	0.866	0.650	0.905	0.095
libarchive	0.102	0.389	0.047	0.962	0.097	0.290	0.208	0.615	0.622	0.232	0.927
libjpeg-turbo	0.661	0.909	0.815	0.838	0.932	0.426	0.350	0.253	0.430	0.972	0.931
libpng	0.486	0.482	0.574	0.738	0.427	0.406	0.413	0.980	0.987	0.950	0.779
libsndfile	0.086	0.191	0.026	0.275	0.529	0.203	0.133	0.740	-	0.473	-
libssh	0.894	0.861	0.688	0.822	0.015	0.464	0.639	0.787	0.796	0.768	0.910
libtiff-read_rgba	0.113	0.775	0.131	0.162	0.805	0.098	0.040	0.065	-	0.386	0.604
libtiff-tiffcp	0.219	0.454	0.831	0.398	0.952	0.562	0.413	0.455	-	0.182	-
libxml2	0.186	0.995	0.838	0.334	0.989	0.385	0.776	0.809	0.974	0.652	0.403
lua	0.043	0.773	0.644	0.648	0.561	0.633	0.369	0.317	-	0.451	0.000
mp3gain	0.800	0.752	0.509	0.676	0.978	0.526	0.088	0.529	0.220	0.391	-
mp4aac	0.067	0.247	0.026	0.009	0.083	0.023	0.261	0.209	0.987	0.887	-
nm	0.045	0.007	0.155	0.197	0.306	0.135	0.437	0.154	0.718	0.793	-
openssl-1.0.1f	0.188	0.082	0.125	0.563	0.082	0.040	0.217	0.286	0.355	0.301	0.082
openssl-1.0.2d	0.854	0.777	0.391	0.466	0.518	0.770	0.934	0.625	0.962	0.606	0.857
openssl-bignum	0.646	0.220	0.812	0.484	0.502	0.325	0.788	0.799	0.638	0.513	0.667
openssl-x509	0.833	0.961	0.798	0.710	0.223	0.985	0.671	0.513	0.598	0.990	0.323
openthread-ip6	0.713	0.745	0.228	0.673	0.800	0.406	0.279	0.183	0.672	0.276	0.809
openthread-radio	0.195	0.230	0.027	0.493	0.003	0.004	0.011	0.013	0.820	0.904	0.541
pcr2	0.273	0.140	0.041	0.065	0.114	0.041	0.256	0.665	0.634	0.576	0.799
pdfotext	0.015	0.440	0.680	0.758	0.793	0.374	0.243	0.481	0.159	0.841	-
php-exif	0.013	0.871	0.214	0.020	0.347	0.230	0.556	0.129	-	0.911	0.854
php-json	0.811	0.921	0.635	0.498	0.577	0.797	0.379	0.729	-	0.824	0.364
php-parser	0.231	0.067	0.006	0.302	0.521	0.204	0.765	0.162	-	0.059	0.828
php-unserialize	0.243	0.874	0.221	0.348	0.452	0.450	0.408	0.875	-	0.789	0.726
poppler-pdf	0.118	0.156	0.323	0.144	0.701	0.274	0.221	0.149	-	0.134	0.833
proj4	0.112	0.009	0.304	0.438	0.134	0.031	0.191	0.092	0.328	0.122	0.899
re2	0.754	0.464	0.377	0.530	0.753	0.935	0.702	0.381	0.463	0.967	0.978
sqliite	0.501	0.567	0.613	0.502	0.872	0.631	0.639	0.541	0.557	0.667	0.510
sqliite3	0.303	0.840	0.525	0.010	0.722	0.596	0.501	0.873	0.647	0.763	-
tcpdump	0.793	0.353	0.868	0.398	0.993	0.639	0.861	0.845	0.947	0.494	-
tiffsplit	0.233	0.298	0.374	0.527	0.919	0.794	0.473	0.068	0.886	0.343	-
vorbis	0.368	0.920	0.032	0.465	0.639	0.124	0.665	0.844	0.980	0.124	0.925
woff2	0.478	0.197	0.556	0.815	0.855	0.263	0.588	0.713	0.873	0.604	0.635
wpanTund	0.060	0.162	0.130	0.822	0.722	0.171	0.346	0.693	0.722	0.225	0.754

Bold and *Italic* represents that a p -value is higher and lower than 0.05, respectively

The experiment results show that the reward uncertainty of the individual fuzzers used in RCFuzzer mostly followed a Beta distribution. Table 5 shows the p -value of the fuzzers in RCFuzzer for each target program, highlighting that most fuzzers align well with the Beta distribution assumption. For 51% (= 24/47) of the total target programs, all the fuzzers' p -values are greater than 0.05, which means their reward distribution statistically follows the Beta distribution. For the remaining 23 programs, more than 80.6% of the fuzzers exhibited a p -value greater than 0.05, which indicates they are still suitable for Beta distribution-based modeling.

5.2. RQ1. Comparison with Individual Fuzzers

The experiment results show that RCFuzzer achieves higher line coverage and detects more unique crashes than the 11 individual fuzzers. Table 6 shows the average line coverage and rankings of individual fuzzers and RCFuzzer in terms of the achieved line coverage for each target program. *Italic* values in

Table 7: The total number of unique crashes detected by the individual fuzzers and RCFuzzer for each target program

Program	AFL	FairFuzz	AFLFast	MOpt	LearnAFL	LAF-INTEL	RedQueen	Radamsa	Angora	QSYM	libFuzzer	RCFuzzer
boringsssl	0	0	0	0	0	0	0	0	0	0	0	0
c-ares	0	0	0	0	0	0	0	0	0	0	1	1
cflow	1	1	1	2	2	0	1	4	0	0	-	4
exiv2	5	7	5	7	7	0	2	1	12	0	-	16
flvmeta	1	1	1	1	1	1	1	1	1	0	-	1
guetzli	1	1	1	1	1	1	1	1	1	0	1	1
harfbuzz	0	0	0	0	0	0	0	0	2	0	0	0
imginfo	0	0	0	1	1	0	0	0	1	0	-	1
infotocap	4	4	4	4	5	0	0	1	0	0	-	7
jhead	0	0	2	2	1	0	7	2	5	0	-	14
json	1	1	1	1	1	1	1	2	0	0	1	2
lame	0	0	0	0	0	2	0	0	1	0	-	4
lcms	0	0	0	0	0	0	0	0	0	0	0	0
libarchive	0	0	0	0	0	0	0	0	0	0	0	1
libjpeg-turbo	0	0	0	0	0	0	0	0	0	0	0	0
libpng	0	0	0	0	0	0	0	0	0	0	0	0
libsndfile	5	6	4	7	8	6	5	5	-	0	-	8
libssh	0	0	0	0	0	0	0	0	0	0	0	0
libtiff-read-rgba	0	0	0	0	0	0	0	0	-	0	0	0
libtiff-tiffcp	1	2	0	1	2	0	1	0	-	0	-	2
libxml2	0	0	0	0	0	0	0	0	0	0	1	1
lua	0	0	0	1	1	0	0	6	-	0	0	5
mp3gain	4	8	3	7	7	2	1	6	6	0	-	9
mp4aac	0	0	0	0	0	0	0	0	3	0	-	2
nm	0	0	0	0	1	0	0	0	0	0	-	4
openssl-1.0.1f	1	0	0	0	0	0	0	0	0	0	1	1
openssl-1.0.2d	0	1	1	1	1	0	0	0	0	0	1	1
openssl-bignum	0	0	0	0	0	0	0	0	0	0	0	0
openssl-x509	0	0	0	0	0	0	0	0	0	0	0	0
openthread-ip6	0	0	0	0	0	0	0	0	0	0	0	0
openthread-radio	0	0	0	0	0	0	0	0	0	0	0	0
pcre2	1	1	1	1	1	1	1	1	1	1	1	1
pdftotext	3	3	2	3	6	3	2	5	2	0	-	10
php-exif	0	0	0	0	0	0	1	0	-	0	3	1
php-json	0	0	0	0	0	0	0	0	-	0	0	0
php-parser	0	0	0	0	0	0	0	0	-	0	0	0
php-unserialize	0	0	0	0	0	0	0	0	-	0	0	0
poppler-pdf	1	2	2	2	2	1	1	5	-	0	1	2
proj4	0	0	0	0	0	0	0	0	0	0	0	0
re2	0	1	0	0	1	1	1	1	0	0	0	1
sqlite	0	0	0	0	0	0	0	0	0	0	0	0
sqlite3	0	0	0	0	0	0	0	0	0	0	-	0
tcpdump	0	0	0	1	2	0	0	0	3	0	-	2
tiffsplit	3	4	4	4	4	3	3	2	4	0	-	7
vorbis	0	0	0	0	0	0	0	0	0	0	0	0
woff2	0	0	0	0	0	1	1	0	0	0	1	1
wpantund	0	0	0	0	0	0	0	0	0	0	0	0
Total	32	44	32	47	55	23	30	43	42	0	12	110

running a individual fuzzer alone.

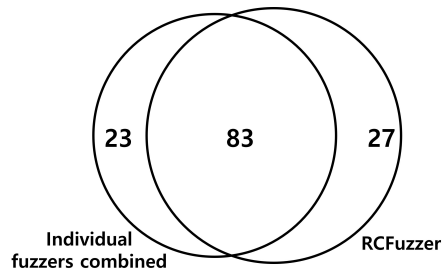


Figure 4: The relation of the unique crashes detected by the individual fuzzers combined and RCFuzzer

Table 7 shows the number of unique crashes detected by the individual fuzzers and RCFuzzer for each target program. RCFuzzer detected about two times more unique crashes than the best-performing individual fuzzers Lear-

Table 8: The memory usage(MB) of the individual fuzzers and RCFuzzer for each target program

Program	AFL	FairFuzz	AFLFast	MOpt	LearnAFL	LAF-INTEL	RedQueen	Radamsa	Angora	QSYM	libFuzzer	RCFuzzer
boringsssl	60.2	61.3	61.2	60.3	61.2	61.2	61.3	61.2	61.3	60.3	61.3	260.4
c-ares	60.0	61.1	61.1	60.1	61.2	61.1	61.1	61.1	61.1	60.1	61.1	258.8
cflow	61.3	61.4	61.3	61.4	61.3	61.3	61.3	61.3	61.4	61.3	-	241.4
exiv2	61.2	61.2	61.2	61.2	61.2	60.2	61.2	61.2	61.4	61.2	-	240.6
flvmeta	61.2	61.4	61.3	61.3	61.3	61.2	61.2	61.2	61.3	60.3	-	241.3
guetzli	60.3	61.3	61.4	60.3	61.3	61.3	61.3	61.3	61.3	60.3	61.3	261.1
harfbuzz	60.3	61.3	61.3	60.4	61.3	61.3	61.3	61.3	61.3	60.3	61.3	260.5
imginfo	61.0	61.2	61.1	61.0	61.1	61.0	61.1	61.1	61.2	61.1	-	239.2
infotocap	60.3	61.4	61.4	61.3	61.4	61.3	61.3	61.2	61.4	60.3	-	241.4
jhead	61.3	61.4	61.4	61.3	61.3	61.3	61.3	61.3	61.3	61.4	-	241.6
json	61.3	61.3	61.3	61.3	61.3	61.2	61.3	61.3	61.3	61.3	61.3	261.0
lame	61.0	61.1	61.1	61.1	61.0	60.1	61.1	61.0	61.2	61.1	-	238.8
lcms	61.2	61.3	61.3	61.2	61.2	61.2	61.2	61.2	61.3	61.2	61.3	259.2
libarchive	61.3	60.3	60.3	61.3	60.3	60.2	60.2	60.2	60.3	61.3	60.3	259.7
libjpeg-turbo	61.3	60.3	60.3	61.3	60.3	60.3	60.3	60.3	60.3	61.3	60.3	261.2
libpng	60.3	61.3	61.3	60.4	61.3	61.3	61.3	61.3	61.4	60.4	61.4	260.4
libsndfile	60.1	60.2	60.2	60.2	60.3	60.3	60.3	61.3	-	60.2	-	219.9
libssh	60.1	61.2	61.2	60.1	61.1	61.1	61.1	61.1	61.1	60.1	61.3	258.7
libtiff-read-rgba	61.3	61.3	61.3	61.3	61.3	61.3	61.3	61.2	-	61.3	61.3	239.4
libtiff-tiffcp	61.3	61.4	61.3	61.3	61.3	61.2	61.3	61.3	-	61.3	61.1	220.6
libxml2	60.3	61.3	61.3	60.3	61.3	61.3	61.3	61.3	61.3	60.3	61.4	260.1
lua	61.3	61.4	61.1	61.3	61.3	61.3	61.3	61.3	-	61.3	61.1	240.2
mp3gain	61.3	61.4	61.3	61.3	61.3	61.2	61.3	61.3	61.1	61.2	-	240.5
mp4aac	61.3	61.3	61.4	61.3	61.3	61.4	61.3	61.3	61.3	61.3	-	240.5
nm	61.3	61.4	61.3	61.4	61.3	61.2	61.3	61.2	61.4	61.1	-	239.6
openssl-1.0.1f	61.1	61.2	61.2	61.1	61.1	61.1	61.1	61.1	61.2	61.3	61.3	258.3
openssl-1.0.2d	61.3	61.4	61.4	61.3	61.3	61.4	61.3	61.3	61.4	61.2	61.3	260.6
openssl-bignum	61.1	61.1	61.0	61.1	61.1	61.3	61.0	61.1	61.0	61.0	61.1	258.9
openssl-x509	61.0	60.1	60.1	61.1	60.1	60.3	60.1	60.1	60.3	61.0	60.2	259.3
openthread-ip6	61.1	61.2	61.1	61.2	61.2	61.1	61.1	61.0	61.0	61.1	61.2	258.0
openthread-radio	61.2	61.4	61.3	61.2	61.3	61.3	61.3	61.3	61.3	61.3	61.3	260.3
pcre2	60.2	61.4	60.3	60.3	60.3	60.3	60.3	60.3	61.3	60.3	60.5	260.8
pdftotext	60.4	61.4	61.4	61.3	61.4	61.3	61.3	61.3	61.4	61.3	-	240.9
php-exif	61.1	61.2	61.2	61.1	61.2	61.2	61.2	61.2	-	60.1	61.4	239.5
php-json	60.2	61.3	61.4	60.3	61.2	61.3	61.3	61.4	-	60.2	61.4	240.9
php-parser	60.3	60.4	61.3	60.3	60.2	60.3	60.3	60.3	-	60.2	61.4	239.5
php-unserialize	61.2	61.2	61.3	61.2	61.1	61.2	61.2	61.2	-	61.1	61.3	239.1
poppler-pdf	60.3	61.3	61.4	60.4	60.3	60.3	61.3	60.3	-	60.3	61.3	238.9
proj4	61.2	61.3	61.2	61.2	61.2	61.2	61.2	61.2	61.1	61.2	61.4	259.2
re2	61.2	61.4	61.4	61.3	61.3	61.3	61.3	61.3	61.3	61.3	61.3	260.6
sqlite	60.3	61.3	61.4	60.3	61.3	61.3	61.3	61.3	61.3	60.3	61.3	260.3
sqlite3	60.2	61.2	61.3	61.2	61.3	61.2	61.2	61.3	61.3	60.3	-	240.3
tcpdump	60.3	61.3	61.3	60.2	61.3	61.3	61.3	61.4	61.3	61.3	-	240.7
tiffsplit	61.3	61.4	61.4	60.3	61.3	61.3	61.2	61.3	61.3	61.3	-	240.3
vorbis	60.3	61.4	61.4	60.4	61.4	61.3	61.3	61.3	61.3	60.4	61.3	260.3
woff2	61.0	61.1	61.1	61.1	61.1	61.3	61.3	61.3	61.0	60.4	61.3	258.8
wpantund	60.3	61.3	61.1	60.3	61.3	61.3	61.1	61.2	61.3	60.1	61.3	260.0
Average	60.8	61.2	61.2	60.9	61.1	61.0	61.1	61.1	61.2	60.8	61.2	249.4

nAFL. Furthermore, RCFuzzer detected unique crashes in the `libarchive` program that were not detected by the 11 individual fuzzers.

The Venn diagram in Figure 4 shows the number of unique crashes detected by the individual fuzzers combined and RCFuzzer. The left circle represents the crashes detected by at least one of the individual fuzzers and the right circle represents the crashes detected by RCFuzzer. RCFuzzer detected 27 unique crashes that no individual fuzzer could detect. Although RCFuzzer did not detect 23 unique crashes found by at least one of the individual fuzzers, it is important to note that the execution time of the individual fuzzers combined is 11 times greater than that of RCFuzzer.

Table 8 presents the memory usage of each individual fuzzer and RCFuzzer across the target programs. On average, RCFuzzer consumes 249.4 MB, approximately four times more than the individual fuzzers. Although RCFuzzer requires more memory, the usage remains within an acceptable range, and it outperforms the individual fuzzers in terms of code coverage and crash detection effectiveness.

Table 9: The average numbers of lines covered, the total number of unique crashes detected, and the memory usage of the state-of-the-art collaborative fuzzer and RCFuzzer

Program	autofz			RCFuzzer			Program	autofz			RCFuzzer		
	Cov	Uniq. crash	Mem (MB)	Cov	Uniq. crash	Mem (MB)		Cov	Uniq. crash	Mem (MB)	Cov	Uniq. crash	Mem (MB)
boringssl	<i>3527.3</i>	1	260.0	3568.7	0	260.4	nm	4385.2	1	239.8	4232.4	4	239.6
c-ares	81.0	1	259.7	81.0	1	258.8	openssl-1.0.1f	9412.2	1	258.7	9134.9	1	258.3
cflow	1772.3	5	241.2	1775.7	4	241.4	openssl-1.0.2d	1943.0	1	260.8	1943.0	1	260.6
exiv2	5213.0	16	240.1	5405.3	16	240.6	openssl-bignum	1777.0	0	258.9	1777.0	0	258.9
flvmeta	625.0	1	240.8	625.0	1	241.3	openssl-x509	9371.2	0	258.7	9371.2	0	259.3
guetzli	4354.5	1	261.4	4351.1	1	261.1	openthread-ipp6	4789.2	0	258.4	4304.5	0	258.0
harfbuzz	6581.9	2	261.4	6594.0	0	260.5	openthread-radio	<i>8852.4</i>	0	260.5	8952.5	0	260.3
imginfo	<i>3280.6</i>	0	240.0	3531.9	1	239.2	pcr2	5911.5	1	260.6	5879.0	1	260.8
infotocap	1859.3	7	240.1	1899.7	7	241.4	pdfotext	<i>10351.9</i>	5	240.7	10481.9	10	240.9
jhead	649.1	13	241.3	647.2	14	241.6	php-exif	<i>26159.8</i>	2	240.8	26181.0	1	239.5
json	2013.0	2	261.5	2012.5	2	261.0	php-json	<i>25513.0</i>	0	241.9	25516.6	0	240.9
lame	1972.7	3	240.1	3036.4	4	238.8	php-parser	<i>30147.4</i>	0	239.8	30588.7	0	239.5
lcms	3898.0	0	259.3	3903.9	0	259.2	php-unserialize	25807.0	0	239.5	25797.2	0	239.1
libarchive	8475.7	0	260.2	8597.0	1	259.7	poppler-pdf	<i>40491.8</i>	1	238.9	41045.9	2	238.9
libjpeg-turbo	3191.3	0	261.8	3207.7	0	261.2	proj4	<i>9779.1</i>	0	258.2	10042.4	0	259.2
libpng	1985.0	0	261.6	1987.0	0	260.4	re2	5459.1	1	259.9	5457.3	1	260.6
libsndfile	6733.7	6	220.9	6779.0	8	219.9	sqlite	2800.0	0	260.1	3282.7	0	260.3
libssh	2498.0	0	259.4	2636.2	0	258.7	sqlite3	1666.9	0	240.3	1668.2	0	240.3
libtiff-read_rgba	<i>6230.7</i>	0	240.9	6381.3	0	239.4	tcpdump	1108.0	0	240.3	1117.1	2	240.7
libtiff-tiffcp	418.0	0	220.2	418.0	2	220.6	tiffsplit	2925.2	8	240.2	2895.9	7	240.3
libxml2	5008.2	0	261.4	5235.7	1	260.1	vorbis	2377.8	0	259.9	2379.7	0	260.3
lua	<i>7087.8</i>	0	240.8	7764.4	5	240.2	woff2	3098.8	1	258.8	3140.2	1	258.8
mp3gain	2156.6	9	240.1	2123.8	9	240.5	wpantund	7370.4	0	258.9	7314.3	0	260.0
mp4aac	3830.2	3	240.5	3861.8	2	240.5							
Average										249.6			249.4
Total										92			110

Cov means the average number of lines covered, **uniq.crash** means the total number of unique crashes detected, and **Mem** means the average number of memory usage(MB). **Bold** text indicates the item with better performance in comparison. *Italic* text indicates a statistically significant difference from RCFuzzer (Mann-Whitney U test, $p < 0.05$).

5.3. RQ2. Comparison with the State-of-the-art Collaborative Fuzzer

The experiment results show that RCFuzzer achieves higher line coverage and detects more unique crashes than the state-of-the-art collaborative fuzzer, autofz. Table 9 shows the average line coverage, the number of unique crashes detected and the memory usage for each target program by autofz and RCFuzzer. RCFuzzer achieved higher line coverage than autofz in 29 out of 47 programs. Notably, for **lame**, RCFuzzer covered more than 1,000 code lines than autofz. *Italic* values in table indicate statistically significant differences ($p < 0.05$) between autofz and RCFuzzer. we used the Mann-Whitney U test to show the statistical significance.

Also, the experiment results show that RCFuzzer has higher bug detection capability than autofz. RCFuzzer detected more unique crashes in 12 programs than autofz. Among these, RCFuzzer detected unique crashes in 6 programs (**imginfo**, **libarchive**, **libtiff-tiffcp**, **lua**, **libxml2**, and **tcpdump**) from which autofz did not detect any crash. RCFuzzer detected unique crashes in approximately twice as many programs as autofz. Comparing the total number of unique crashes detected, RCFuzzer identified 18 more unique crashes than autofz. Additionally, the Venn diagram in Figure 5 shows the distribution of the unique crashes detected by autofz and RCFuzzer across all programs. According to the diagram, RCFuzzer detected 30 unique crashes that autofz could not detect.

This demonstrates that RCFuzzer can detect more bugs and explore more

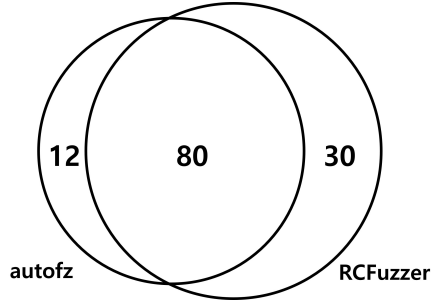


Figure 5: The relation of the unique crashes detected by the state-of-the-art collaborative fuzzer(autofz) and RCFuzzer

paths with the same amount of time than the state-of-the-art collaborative fuzzer like autofz³. This demonstrates that the thompson sampling method of selecting the most efficient fuzzer over time is more effective than comparing the performance of fuzzers in each round and allocating additional resources to a specific fuzzer.

In terms of memory usage, both of autofz and RCFuzzer consume around 250MB.

5.4. RQ3. Effects of the Collection Phase

The experiment results show that the collection phase is effective to improve code coverage and crash detection capability. RCFuzzer achieves higher line coverage and detects significantly more unique crashes than *RCFuzzer^{Collect-}*. Table 10 shows the average line coverage, the number of unique crashes detected, and the memory usage for each target program by *RCFuzzer^{Collect-}* and RCFuzzer. RCFuzzer achieved higher line coverage for 34 out of 47 programs than *RCFuzzer^{Collect-}*. Notably, for *lame*, RCFuzzer covered more than 1,500 code lines than *RCFuzzer^{Collect-}*. Italic values in table indicate statistically significant differences ($p < 0.05$) between *RCFuzzer^{Collect-}* and RCFuzzer. We used the Mann–Whitney U test to show the statistical significance.

Also, RCFuzzer detected more unique crashes than *RCFuzzer^{Collect-}* in 14 programs. Specifically, RCFuzzer detected unique crashes in three programs (*libarchive*, *re2*, and *tcpdump*) from which *RCFuzzer^{Collect-}* could not detect. Overall, RCFuzzer detected 25 more unique crashes than *RCFuzzer^{Collect-}*. The improvement of crash detection by the collection phase is approximately $29.4\% = (110 - 85) / 85 * 100$.

In terms of memory usage, both of *RCFuzzer^{Collect-}* and RCFuzzer consume around 250MB.

³We did not compare RCFuzzer with CUPID (Güler et al., 2020) and ENFUZZ (Chen et al., 2019) because the authors of autofz (Fu et al., 2023) showed that autofz already outperformed CUPID and ENFUZZ

Table 10: The average numbers of lines covered, the total number of unique crashes detected, and the memory usage of RCFuzzer with and without the collection phase

Program	RCFuzzer ^{Collect-}			RCFuzzer			Program	RCFuzzer ^{Collect-}			RCFuzzer		
	Cov	Uniq. crash	Mem (MB)	Cov	Uniq. crash	Mem (MB)		Cov	Uniq. crash	Mem (MB)	Cov	Uniq. crash	Mem (MB)
boringssl	3580.8	0	259.1	3568.7	0	260.4	nm	4236.8	2	239.6	4232.4	4	239.6
c-ares	81.0	1	259.5	81.0	1	258.8	openssl-1.0.1f	<i>8900.9</i>	1	259.0	9134.9	1	258.3
cflow	1769.3	3	240.7	1775.7	4	241.4	openssl-1.0.2d	1943.0	1	260.5	1943.0	1	260.6
exiv2	<i>5023.1</i>	11	240.2	5405.3	16	240.6	openssl-bignum	1777.0	0	258.2	1777.0	0	258.9
flvmeta	625.0	1	240.7	625.0	1	241.3	openssl-x509	9371.2	0	259.3	9371.2	0	259.3
guetzli	4350.4	1	260.6	4351.1	1	261.1	openthread-ip6	4295.8	0	258.8	4304.5	0	258.0
harfbuzz	<i>6497.1</i>	1	259.7	6594.0	0	260.5	openthread-radio	<i>8852.3</i>	0	260.1	8952.5	0	260.3
imginfo	3331.1	1	238.8	3531.9	1	239.2	pcr2	5895.6	1	260.2	5879.0	1	260.8
infotocap	1848.3	6	240.6	1899.7	7	241.4	pdftotext	10472.0	11	239.7	10481.9	10	240.9
jhead	<i>549.2</i>	6	240.3	647.2	14	241.6	php-exif	<i>26156.2</i>	1	239.2	26181.0	1	239.5
json	2010.8	1	261.1	2012.5	2	261.0	php-json	<i>25513.1</i>	0	239.6	25516.6	0	240.9
lame	1463.2	3	239.1	3036.4	4	238.8	php-parser	30591.2	0	238.8	30588.7	0	239.5
lcms	<i>3796.4</i>	0	260.2	3903.9	0	259.2	php-unserialize	25795.8	0	238.1	25797.2	0	239.1
libarchive	8392.1	0	259.9	8597.0	1	259.7	poppler-pdf	<i>40599.9</i>	3	238.1	41045.9	2	238.9
libjpeg-turbo	3114.7	0	260.7	3207.7	0	261.2	proj4	10025.8	0	258.3	10042.4	0	259.2
libpng	1947.3	0	260.2	1987.0	0	260.4	re2	5452.0	0	259.9	5457.3	1	260.6
libsndfile	<i>6396.5</i>	8	218.4	6779.0	8	219.9	sqlite	2793.0	0	259.6	3282.7	0	260.3
libssh	2570.2	0	259.3	2636.2	0	258.7	sqlite3	1667.6	0	239.6	1668.2	0	240.3
libtiff-read_rgba	6291.1	0	238.7	6381.3	0	239.4	tcpdump	1113.9	0	239.7	1117.1	2	240.7
libtiff-tiffcp	418.0	1	218.5	418.0	2	220.6	tiffsplit	2864.6	6	239.5	2895.9	7	240.3
libxml2	5478.8	1	258.9	5235.7	1	260.1	vorbis	2380.3	0	258.6	2379.7	0	260.3
lua	7454.7	3	238.3	7764.4	5	240.2	woff2	3128.5	1	257.8	3140.2	1	258.8
mp3gain	2112.6	8	239.6	2123.8	9	240.5	wpantund	7345.5	0	257.8	7314.3	0	260.0
mp4aac	3828.0	2	239.9	3861.8	2	240.5							
Average										248.8			249.4
Total									85				110

Cov means the average number of line covered, **uniq.crash** means the total number of unique crashes detected, and **Mem** means the average number of memory usage(MB). **Bold** text indicates the item with better performance in comparison. *Italic* text indicates a statistically significant difference from RCFuzzer (Mann-Whitney U test, $p < 0.05$).

5.5. RQ4. Effects of the Recommendation-based Fuzzer Selection

The experiment results show that recommendation-based fuzzer selection is effective to improve code coverage and crash detection capability. RCFuzzer achieves higher line coverage and detects significantly more unique crashes than RCFuzzer with random selection instead of recommendation-based fuzzer selection. Table 11 shows the average line coverage, the number of unique crashes detected, and the memory usage for each target program of *RCFuzzer^{Rand}* and RCFuzzer. RCFuzzer achieved higher line coverage in 37 out of 47 programs than *RCFuzzer^{Rand}*. Notably, for **lame**, RCFuzzer covered more than 1,500 code lines than *RCFuzzer^{Rand}*. Italic values in table indicate statistically significant differences ($p < 0.05$) between *RCFuzzer^{Rand}* and RCFuzzer. We used the Mann-Whitney U test to show the statistical significance.

In terms of unique crashes, RCFuzzer detected more unique crashes in 11 out of 47 programs than *RCFuzzer^{Rand}*. RCFuzzer detected unique crashes in five programs (**imginfo**, **libarchive**, **libtiff-tiffcp**, **lua** and **nm**) from which *RCFuzzer^{Rand}* could not detect. For the remaining 36 programs, both RCFuzzer and *RCFuzzer^{Rand}* identified the same number of unique crashes. Overall, RCFuzzer detected 31 more unique crashes than *RCFuzzer^{Rand}*. The improvement of bug detection capability by the recommendation-based fuzzer selection is approximately 39.2% ($= (110 - 79) / 79 * 100$).

In terms of memory usage, both of *RCFuzzer^{Rand}* and RCFuzzer consume around 250MB.

Table 11: The average numbers of lines covered, the total number of unique crashes detected, and the memory usage of RCFuzzer and RCFuzzer with random selection instead of recommendation-based selection.

Program	<i>RCFuzzer^{Rand}</i>			RCFuzzer			Program	<i>RCFuzzer^{Rand}</i>			RCFuzzer		
	Cov	Uniq. crash	Mem (MB)	Cov	Uniq. crash	Mem (MB)		Cov	Uniq. crash	Mem (MB)	Cov	Uniq. crash	Mem (MB)
boringsssl	<i>3505.2</i>	0	260.8	3568.7	0	260.4	nm	4223.3	0	241.1	4232.4	4	239.6
c-ares	81.0	1	259.4	81.0	1	258.8	openssl-1.0.1f	9049.9	1	259.3	9134.9	1	258.3
cflow	1771.4	4	242.1	1775.7	4	241.4	openssl-1.0.2d	1943.0	1	261.6	1943.0	1	260.6
exiv2	<i>4714.2</i>	10	240.1	5405.3	16	240.6	openssl-bignum	1777.0	0	259.6	1777.0	0	258.9
flvmeta	625.0	1	241.4	625.0	1	241.3	openssl-x509	9371.0	0	260.1	9371.2	0	259.3
guetzli	4353.2	1	260.7	4351.1	1	261.1	openthread-ip6	4229.9	0	259.5	4304.5	0	258.0
harfbuzz	<i>6495.3</i>	0	261.0	6594.0	0	260.5	openthread-radio	<i>8626.7</i>	0	260.9	8952.5	0	260.3
imginfo	<i>3227.6</i>	0	239.8	3531.9	1	239.2	pcr2	5877.4	1	261.7	5879.0	1	260.8
infotocap	<i>1754.2</i>	3	241.5	1899.7	7	241.4	pdfotext	<i>10149.9</i>	5	241.9	10481.9	10	240.9
jhead	<i>635.9</i>	13	241.2	647.2	14	241.6	php-exif	<i>26127.4</i>	1	240.4	26181.0	1	239.5
json	2012.8	2	261.2	2012.5	2	261.0	php-json	<i>25511.3</i>	0	241.6	25516.6	0	240.9
lame	1472.1	3	239.7	3036.4	4	238.8	php-parser	<i>29558.1</i>	0	240.8	30588.7	0	239.5
lcms	3856.6	0	259.7	3903.9	0	259.2	php-unserialize	<i>25791.1</i>	0	239.4	25797.2	0	239.1
libarchive	<i>8141.4</i>	0	261.0	8597.0	1	259.7	poppler-pdf	<i>40266.7</i>	2	239.3	41045.9	2	238.9
libjpeg-turbo	<i>3162.1</i>	0	261.9	3207.7	0	261.2	proj4	<i>9377.9</i>	0	260.6	10042.4	0	259.2
libpng	1993.6	0	260.8	1987.0	0	260.4	rs2	5435.8	1	261.8	5457.3	1	260.6
libpng	<i>6576.8</i>	8	220.3	6779.0	8	219.9	sqliite	2822.1	0	260.9	3282.7	0	260.3
libssh	<i>2155.2</i>	0	259.5	2636.2	0	258.7	sqliite3	1667.6	0	240.4	1668.2	0	240.3
libtiff-read-rgba	<i>6098.4</i>	0	241.1	6381.3	0	239.4	tcpdump	1116.2	1	242.4	1117.1	2	240.7
libtiff-tiffcp	418.0	0	220.7	418.0	2	220.6	tiffsplit	<i>2809.0</i>	7	241.3	2895.9	7	240.3
libxml2	<i>5046.9</i>	1	260.6	5235.7	1	260.1	vorbis	<i>2377.7</i>	0	261.2	2379.7	0	260.3
lua	7485.8	0	240.4	7764.4	5	240.2	woff2	3077.6	1	259.6	3140.2	1	258.8
mp3gain	2146.8	9	241.5	2123.8	9	240.5	wpantund	7347.4	0	261.1	7314.3	0	260.0
mp4aac	<i>3635.3</i>	2	241.6	3861.8	2	240.5							
Average										250.1			249.4
Total									79			110	

Cov means the average number of lines covered, **Uniq.crash** means the total number of unique crashes detected, and **Mem** means the average number of memory usage(MB). **Bold** text indicates the item with better performance in comparison. *Italic* text indicates a statistically significant difference from RCFuzzer (Mann-Whitney U test, $p < 0.05$).

5.6. RQ5. Effects of the Branch Difficulty-based Heuristic

The experiment results show that RCFuzzer achieves higher line coverage and detects significantly more unique crashes than running RCFuzzer without the branch difficulty-based Heuristic. Table 12 shows the average line coverage, the number of unique crashes detected, and the memory usage for each target program of *RCFuzzer^{Diff}* and RCFuzzer. RCFuzzer achieved higher line coverage in 27 out of 47 programs than *RCFuzzer^{Diff}*. Notably, for **lame**, RCFuzzer covered approximately more than 1,400 code lines than *RCFuzzer^{Diff}*. Italic values in table indicate statistically significant differences ($p < 0.05$) between *RCFuzzer^{Diff}* and RCFuzzer. We used the Mann-Whitney U test to show the statistical significance.

In terms of unique crashes, RCFuzzer detected more unique crashes in 13 programs than *RCFuzzer^{Diff}*, with RCFuzzer detecting unique crashes in two programs (**libarchive** and **libxml2**) from which *RCFuzzer^{Diff}* could not detect. Additionally, RCFuzzer detected more unique crashes in 11 target programs compared to *RCFuzzer^{Diff}*. Overall, RCFuzzer detected 17 more unique crashes than *RCFuzzer^{Diff}*. The bug detection capability improvement is approximately 18.3% ($= (110-93)/93 * 100$).

In terms of memory usage, both of *RCFuzzer^{Diff}* and RCFuzzer consume around 250MB.

Table 12: The average numbers of lines covered, the total number of unique crashes detected, and the memory usage of RCFuzzer with and without the branch difficulty-based heuristic

Program	RCFuzzer ^{Diff}			RCFuzzer			Program	RCFuzzer ^{Diff}			RCFuzzer		
	Cov	Uniq. crash	Mem (MB)	Cov	Uniq. crash	Mem (MB)		Cov	Uniq. crash	Mem (MB)	Cov	Uniq. crash	Mem (MB)
boringsssl	<i>3521.1</i>	0	261.0	3568.7	0	260.4	nm	4496.0	2	240.9	4232.4	4	239.6
c-ares	81.0	1	259.1	81.0	1	258.8	openssl-1.0.1f	9522.2	1	259.0	9134.9	1	258.3
cflow	1770.9	3	241.9	1775.7	4	241.4	openssl-1.0.2d	1943.0	1	261.7	1943.0	1	260.6
exiv2	<i>4903.8</i>	15	240.1	5405.3	16	240.6	openssl-bignum	1777.0	0	259.3	1777.0	0	258.9
flvmeta	625.0	1	241.1	625.0	1	241.3	openssl-x509	9371.0	0	258.8	9371.2	0	259.3
guetzli	4353.0	1	260.6	4351.1	1	261.1	openthread-ip6	4452.2	0	259.4	4304.5	0	258.0
harfbuzz	<i>6508.8</i>	1	260.9	6594.0	0	260.5	openthread-radio	<i>8840.6</i>	0	260.3	8952.5	0	260.3
imginfo	3507.9	1	239.2	3531.9	1	239.2	pcr2	5883.9	1	261.4	5879.0	1	260.8
infotocap	1834.1	6	241.4	1899.7	7	241.4	pdfotext	10344.9	5	240.9	10481.9	10	240.9
jhead	623.3	15	241.0	647.2	14	241.6	php-exif	26166.5	1	239.9	26181.0	1	239.5
json	2013.0	2	261.3	2012.5	2	261.0	php-json	25514.5	0	241.2	25516.6	0	240.9
lame	1636.7	1	239.5	3036.4	4	238.8	php-parser	30560.2	0	240.1	30588.7	0	239.5
lcms	3911.8	0	259.5	3903.9	0	259.2	php-unserialize	25793.8	0	239.4	25797.2	0	239.1
libarchive	8665.9	0	260.4	8597.0	1	259.7	poppler-pdf	40954.0	4	238.2	41045.9	2	238.9
libjpeg-turbo	3193.3	0	261.6	3207.7	0	261.2	proj4	10082.8	0	259.8	10042.4	0	259.2
libpng	1989.7	0	260.5	1987.0	0	260.4	re2	5452.2	1	261.6	5457.3	1	260.6
libsndfile	6714.2	8	220.4	6779.0	8	219.9	sqlite	2793.0	0	260.6	3282.7	0	260.3
libssh	2507.5	0	259.2	2636.2	0	258.7	sqlite3	1668.2	0	240.2	1668.2	0	240.3
libtiff-read_rgba	6303.3	0	241.0	6381.3	0	239.4	tcpdump	1113.7	1	242.1	1117.1	2	240.7
libtiff-tiffcp	418.0	1	220.0	418.0	2	220.6	tiffsplit	2886.7	6	240.1	2895.9	7	240.3
libxml2	5302.2	0	259.9	5235.7	1	260.1	vorbis	2379.1	0	261.2	2379.7	0	260.3
lua	<i>7287.7</i>	2	238.7	7764.4	5	240.2	woff2	3117.7	1	259.6	3140.2	1	258.8
mp3gain	2153.1	8	240.6	2123.8	9	240.5	wpantund	7342.6	0	260.4	7314.3	0	260.0
mp4aac	3872.4	3	240.7	3861.8	2	240.5							
Average								249.7			249.4		
Total								93			110		

* **Cov** means the average number of lines covered, **uniq.crash** means the total number of unique crashes detected, and **Mem** means the average number of memory usage(MB). **Bold** text indicates the item with better performance in comparison. *Italic* text indicates a statistically significant difference from RCFuzzer (Mann-Whitney U test, $p < 0.05$).

6. DISCUSSION

6.1. Synergy Effects of Collaborative Fuzzing

Table 13: The number of the unique crashes detected by the best individual fuzzer for each program and RCFuzzer

Program	Commonly detects	Only individual fuzzer detected	Only RCFuzzer detects	Program	Commonly detects	Only individual fuzzer detected	Only RCFuzzer detects
boringsssl	0	0	0	nm	0	1	4
c-ares	1	0	0	openssl-1.0.1f	0	1	0
cflow	4	0	0	openssl-1.0.2d	1	0	0
exiv2	9	3	7	openssl-bignum	0	0	0
flvmeta	1	0	0	openssl-x509	0	0	0
guetzli	1	0	0	openthread-ip6	0	0	0
harfbuzz	0	2	0	openthread-radio	0	0	0
imginfo	1	0	0	pcr2	0	1	1
infotocap	5	0	2	pdfotext	6	0	4
jhead	7	0	7	php-exif	1	2	0
json	2	0	0	php-json	0	0	0
lame	2	0	2	php-parser	0	0	0
lcms	0	0	0	php-unserialize	0	0	0
libarchive	0	0	1	poppler-pdf	2	3	0
libjpeg-turbo	0	0	0	proj4	0	0	0
libpng	0	0	0	re2	1	0	0
libsndfile	8	0	0	sqlite	0	0	0
libssh	0	0	0	sqlite3	0	0	0
libtiff-read_rgba	0	0	0	tcpdump	0	3	2
libtiff-tiffcp	2	0	0	tiffsplit	4	0	3
libxml2	0	1	1	vorbis	0	0	0
lua	5	1	0	woff2	1	0	0
mp3gain	8	0	1	wpantund	0	0	0
mp4aac	2	1	0				
Total					75	18	35

In this study, we found that the synergy effects of collaborative fuzzing. Collaborative fuzzing not only finds the optimal fuzzer for a given target program, but it does fuzzing better than the best individual fuzzer for the given target program.

As shown in Table 13, when comparing the number of unique crashes detected by the best individual fuzzer for each program and RCFuzzer, RCFuzzer demonstrated the ability to detect 17 more unique crashes than the best individual fuzzer. In particular, an analysis of programs such as `infotocap`, `jhead`, `mp3gain`, `pdftotext`, and `tiffsplit` showed that RCFuzzer not only detected all the unique crashes found by individual fuzzers but also identified additional unique crashes that individual fuzzers failed to discover. This indicates that collaborative fuzzing is not merely a combination of fuzzers but rather exhibits a powerful synergy effect that enables achievements unattainable by individual fuzzers alone.

In conclusion, the synergy effect of collaborative fuzzing has established it as a powerful tool capable of detecting more vulnerabilities in diverse environments. These results prove that collaborative fuzzing is a highly effective approach in increasingly complex software environments and suggest that there is significant value in further developing this method in the future.

6.2. Comparison of Fuzzer Selection Mechanisms: *autofz* vs *RCFuzzer*

RCFuzzer is more effective than *autofz* (Section 5.3) because RCFuzzer reduces waste of time in the fuzzer selection process.

The fuzzing process of *autofz* (Fu et al., 2023) consists of a preparation phase and a focus phase. In the preparation phase, all fuzzers are executed equally, and achieved code coverage is compared. The preparation phase can be time-consuming, leading to inefficiencies. *autofz* tries to avoid such waste of time through early termination of the preparation phase. If the difference between the highest and lowest coverage exceeds a threshold, early termination occurs, and the fuzzer with the highest coverage is selected for the focus phase. However, such early termination rarely occurs as fuzzing time goes on and the covering uncovered code becomes difficult.

In contrast, RCFuzzer uses a collection phase to collect the performance of each fuzzer using the same seed corpus each time. In the subsequent execution phase, fuzzers are selected and executed using the same seed corpus, ensuring that the chosen fuzzer is always expected to be the most effective. Because fuzzer selection is based on previous performance data, no additional time is needed, resulting in less wasted time compared to *autofz*.

Figure 6 is a graph comparing the fuzzer selection scheduling of *autofz* and RCFuzzer for the `Imginfo` program. As shown in the graph, *autofz* exhibited a steady increase in fuzzer performance during the first 9 hours, but this improvement halted afterward. After 10 hours, as no early termination occurred, the fuzzer selection time extended, leading to increased time consumption during the preparation phase and resulting in inefficiencies. In contrast, RCFuzzer explores the performance of various fuzzers during the initial 7 hours, maximizing their effectiveness, and then effectively allocates resources by selecting the

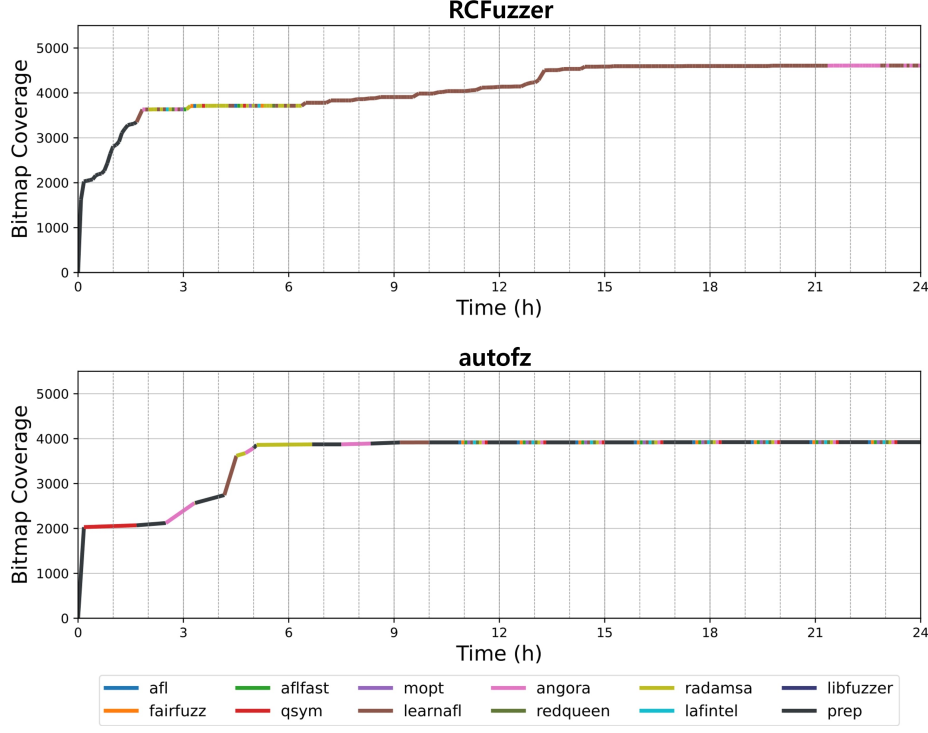


Figure 6: Fuzzer selection scheduling for the `Imginfo` program: Comparing autofz and RCFuzzer

best-performing fuzzer (`learnaf1`) based on the collected performance data. As time progresses, RCFuzzer continues to utilize this data to select the optimal fuzzer, allowing it to achieve optimal results in subsequent phases.

7. RELATED WORK

7.1. Collaborative Fuzzing

Collaborative Fuzzing (Chen et al., 2019; Güler et al., 2020; Fu et al., 2023; Österlund et al., 2021; Li et al., 2023; Shi et al., 2024) is a software testing technique that improves the overall effectiveness of the fuzzing process by leveraging the strengths of multiple fuzzers.

ENFuzz (Chen et al., 2019), the first collaborative fuzzing technique, proposes an ensemble fuzzing approach that enhances bug detection and coverage through seed synchronization among various fuzzers. ENFuzz manually combines multiple fuzzers, configuring each to leverage its unique strengths in coverage and mutation strategies. During this process, it analyzes the characteristics of each fuzzer and finds the optimal combination to collaborate via the seed synchronization mechanism. As a result, ENFuzz demonstrates superior path

coverage and bug detection performance compared to individual fuzzers. Unlike ENFuzz, RCFuzzer does not manually combine the optimal fuzzers but automatically selects and executes the most efficient fuzzer every hour from a variety of fuzzers.

CUPID (Güler et al., 2020) advances the concept proposed by ENFuzz by automatically selecting the most complementary fuzzers for collaborative fuzzing. Using past performance data of individual fuzzers (such as coverage, bug detection rate, execution time, etc.), it employs a machine learning approach to predict and select the most efficient combination of fuzzers. This reduces the overhead of manual fuzzer configuration and optimization, thereby improving coverage and shortening the time to discover bugs. Unlike CUPID, RCFuzzer selects the most efficient fuzzer in run-time while collecting data, without relying on past performance data of each fuzzer.

COLLABFuzz (Österlund et al., 2021) is a collaborative fuzzing framework designed to promote cooperation among fuzzers by sharing information and distributing fuzzing tasks. Multiple fuzzers operate simultaneously, with each fuzzer sharing discovered inputs and feedback so that other fuzzers can utilize this information. This framework supports various test case policies, enabling run-time orchestration and synchronization. By efficiently distributing tasks among fuzzers and dynamically adjusting strategies based on run-time feedback, it minimizes redundant work and optimizes resource usage.

autofz (Fu et al., 2023) analyzes the runtime characteristics of the target program and dynamically selects and executes the most suitable fuzzer or combination of fuzzers. The fuzzing process involves continuous monitoring and automatically selects fuzzers based on run-time feedback. autofz’s fuzzer selection method is determined by the bitmap coverage difference between fuzzers in each round. In contrast, RCFuzzer uses a recommendation-based fuzzer selection method, performing exploration and exploitation based on the performance data of each fuzzer in previous exploration phases.

PyRTFuzz (Li et al., 2023) is a tool that combines generation-based fuzzing and mutation-based fuzzing to comprehensively test the Python interpreter and runtime libraries. This tool extracts runtime API descriptions through static and dynamic analysis and generates Python applications that can test each API using a declarative specification language. These applications utilize custom type-based mutation strategies to generate diverse inputs, overcoming the traditional challenges of handling data types. Through this, PyRTFuzz effectively verifies the security and reliability of the Python runtime. In contrast, RCFuzzer focuses on optimizing the fuzzing process by selecting the optimal fuzzer from various types of fuzzers.

BandFuzz (Shi et al., 2024) is an efficient collaborative fuzzing framework designed to identify software vulnerabilities. By utilizing multiple fuzzers, it evaluates the performance of various fuzzers in run-time and predicts the optimal fuzzer to allocate resources. This approach demonstrates the ability to efficiently manage the fuzzing process even in resource-constrained environments. BandFuzz is still under development, so a detailed comparison is not possible due to the lack of detailed technical information. In contrast, RCFuzzer has

already been developed, and its technical aspects are thoroughly documented.

7.2. Reinforcement Learning and Thompson Sampling based Fuzzer

Several fuzzing technique (Karamcheti et al., 2018; Scott et al., 2020; Lee et al., 2023; Deng et al., 2023; Jha et al., 2024) studies have modeled the problem of optimal selection using reinforcement learning algorithms as a multi-armed bandit problem.

Siddharth Karamcheti et al. (Karamcheti et al., 2018) applied the Thompson Sampling algorithm, which probabilistically estimates the potential rewards of each action in the multi-armed bandit problem, to fuzz testing. By doing so, they learned the effectiveness of input mutation strategies and developed a technique to more frequently select promising strategies. This approach demonstrates the selection of mutation strategies tailored to individual programs.

BanditFuzz (Scott et al., 2020) is a method that leverages multi-armed bandit (MAB) algorithms to enhance the performance testing of SMT solvers. This approach models the test case generator as a reinforcement learning agent, learning to generate test cases that effectively identify performance bottlenecks in SMT solvers. This allows for more efficient evaluation and optimization of solver performance across various inputs compared to traditional random fuzzing methods, showing improvements in SMT solvers.

SEAMFuzz (Lee et al., 2023) leverages reinforcement learning to dynamically select seeds and mutation strategies, optimizing the fuzzing process based on feedback from the software under test. It learns seeds and mutation strategies that are likely to discover new execution paths and potential vulnerabilities. This approach demonstrates high coverage and the detection of numerous crashes in diverse software environments by dynamically selecting effective mutation methods optimized for each seed group.

TitanFuzz (Deng et al., 2023) is a method for discovering defects in deep learning libraries using a pretrained large language model (LLM) (Brown et al., 2020). It generates various patterns and conditions for prompts (Liu et al., 2021) used by the LLM through mutation operators. To learn and select the most effective mutation operator, it models the problem as a multi-armed bandit (MAB) problem and uses the Thompson Sampling algorithm. This allows it to choose the best-performing mutation operator, creating prompts that generate more valid and unique code snippets. This approach demonstrates the ability to effectively identify potential bugs through diverse prompts provided to the large language model.

BertRLFuzzer (Jha et al., 2024) improves the fuzzing process by combining the capabilities of the pre-trained language model BERT (Devlin et al., 2019) with reinforcement learning (RL). This method leverages BERT to generate diverse and meaningful test inputs, creating more effective test cases based on BERT’s deep understanding of language patterns. Subsequently, reinforcement learning is applied to dynamically adjust the fuzzing strategy based on feedback from the software, selecting test cases that are more likely to explore new execution paths and discover vulnerabilities. This approach, through the innovative

combination of natural language processing and reinforcement learning, provides a highly adaptive fuzzing method capable of uncovering various software vulnerabilities.

While existing studies have utilized reinforcement learning to optimize input mutation strategies or seed selection within a specific fuzzer, RCFuzzer employs reinforcement learning to select the optimal fuzzer from a variety of fuzzers. This approach improves the effectiveness of fuzzing by choosing the most effective fuzzer for a given state of the target program.

7.3. Seed Synchronization

Seed synchronization is a crucial technique that enhances the efficiency and effectiveness of fuzzing by enabling the sharing of interesting seeds generated by different instances of the same fuzzer (Chen et al., 2019; Güler et al., 2020; Österlund et al., 2021; Fu et al., 2023; Liang et al., 2018; Sun and Xu, 2023; Zalewski, 2017; Pham et al., 2022; Liang et al., 2024; Wang et al., 2021). This helps fuzzers overcome stagnation and explore new paths and vulnerabilities. Dodrio (Liang et al., 2024) presents another example of seed synchronization by dynamically evaluating the validity of shared seeds among fuzzers to reduce unnecessary redundancy. Recently, the concept of ensemble fuzzing has been introduced through systems like EnFuzz (Chen et al., 2019), Cupid (Güler et al., 2020), CollabFuzz (Österlund et al., 2021), and autofz (Fu et al., 2023) which combine various fuzzers into an ensemble to maximize seed generation capabilities and improve code coverage.

To support large-scale fuzzing, research has focused on parallel fuzzer synchronization. This involves multiple parallel instances of the same fuzzer sharing and synchronizing a single corpus. For instance, PAFL (Liang et al., 2018) synchronizes guiding information and partitions tasks based on a bitmap to avoid duplication, while AFLTeam (Pham et al., 2022) and AFL-EDGE (Wang et al., 2021) use static analysis to divide the call or control flow graph, distributing mutually exclusive tasks to different instances. These approaches enable efficient collaboration between fuzzer instances.

Overall, the practice of seed synchronization has proven to be a valuable strategy in modern fuzzing, facilitating greater collaboration between fuzzer instances and enhancing the detection of software vulnerabilities.

8. CONCLUSION

In this paper, we presented RCFuzzer, the innovative collaborative fuzzing technique that optimizes the selection and execution of the individual fuzzers through one of the recommendation approaches, thompson sampling. In the initial collection phase, performance data from various fuzzers is collected to evaluate the effectiveness of each in discovering vulnerabilities. Based on the collected data, the most promising fuzzer is selected for the execution phase to maximize line coverage and uncover crashes. The empirical evaluation showed that RCFuzzer was more effective in achieving higher line coverage and finding

more bugs compared to the individual fuzzers, and RCFuzzer outperformed the state-of-the-art collaborative fuzzer autofz.

For future research, we plan to improve the performance of RCFuzzer. First, we will evaluate RCFuzzer’s applicability to various types of programs and complexities, and explore ways to optimize the recommendation models accordingly. Additionally, we aim to expand the fuzzer selection algorithm to collaborate with a broader range of fuzzers, creating a more comprehensive collaborative fuzzing system. Finally, we will refine the dynamic adjustment of fuzzers using runtime feedback and research strategies to effectively manage resource constraints during the fuzzing process, thereby enhancing the practicality of RCFuzzer.

Acknowledgments

This work was supported in part by the National Research Foundation of Korea(NRF) grant funded by the Korea government(MSIT) (No. NRF-2021R1A5A1021944), the research fund of Hanyang University(HY-202000000002755), and Samsung Electronics.

References

- Agrawal, S., Goyal, N., 2012. Analysis of thompson sampling for the multi-armed bandit problem. URL: <https://arxiv.org/abs/1111.1797>, arXiv:1111.1797.
- Artin, E., 2015. The gamma function. Courier Dover Publications.
- Aschermann, C., Schumilo, S., Blazytko, T., Gawlik, R., Holz, T., 2019. Redqueen: Fuzzing with input-to-state correspondence., in: NDSS, pp. 1–15.
- Boehme, M., Cadar, C., Roychoudhury, A., 2021. Fuzzing: Challenges and reflections. IEEE Software 38, 79–86. doi:10.1109/MS.2020.3016773.
- Böhme, M., Pham, V.T., Roychoudhury, A., 2016. Coverage-based greybox fuzzing as markov chain, in: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, Association for Computing Machinery, New York, NY, USA. p. 1032–1043. URL: <https://doi.org/10.1145/2976749.2978428>, doi:10.1145/2976749.2978428.
- Brown, T.B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D.M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., Amodei, D., 2020. Language models are few-shot learners. URL: <https://arxiv.org/abs/2005.14165>, arXiv:2005.14165.

- Chapelle, O., Li, L., 2011. An empirical evaluation of thompson sampling, in: Proceedings of the 24th International Conference on Neural Information Processing Systems, Curran Associates Inc., Red Hook, NY, USA. p. 2249–2257.
- Chen, P., Chen, H., 2018. Angora: Efficient fuzzing by principled search, in: 2018 IEEE Symposium on Security and Privacy (SP), pp. 711–725. doi:10.1109/SP.2018.00046.
- Chen, Y., Jiang, Y., Ma, F., Liang, J., Wang, M., Zhou, C., Su, Z., Jiao, X., 2019. Enfuzz: Ensemble fuzzing with seed synchronization among diverse fuzzers. URL: <https://arxiv.org/abs/1807.00182>, arXiv:1807.00182.
- Chiu, D.M., Jain, R., 1989. Analysis of the increase and decrease algorithms for congestion avoidance in computer networks. Computer Networks and ISDN Systems 17, 1–14. URL: <https://www.sciencedirect.com/science/article/pii/0169755289900196>, doi:[https://doi.org/10.1016/0169-7552\(89\)90019-6](https://doi.org/10.1016/0169-7552(89)90019-6).
- Deng, Y., Xia, C.S., Peng, H., Yang, C., Zhang, L., 2023. Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models. URL: <https://arxiv.org/abs/2212.14834>, arXiv:2212.14834.
- Devlin, J., Chang, M.W., Lee, K., Toutanova, K., 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. URL: <https://arxiv.org/abs/1810.04805>, arXiv:1810.04805.
- Fioraldi, A., Mantovani, A., Maier, D., Balzarotti, D., 2023. Dissecting american fuzzy lop: A fuzzbench evaluation. ACM Trans. Softw. Eng. Methodol. 32. URL: <https://doi.org/10.1145/3580596>, doi:10.1145/3580596.
- Fisher, R.A., 1970. Statistical methods for research workers, in: Breakthroughs in statistics: Methodology and distribution. Springer, pp. 66–70.
- Fu, Y.F., Lee, J., Kim, T., 2023. autofz: Automated fuzzer composition at runtime, in: 32nd USENIX Security Symposium (USENIX Security 23), USENIX Association, Anaheim, CA. pp. 1901–1918. URL: <https://www.usenix.org/conference/usenixsecurity23/presentation/fu-yu-fu>.
- Google, a. AddressSanitizer. <https://clang.llvm.org/docs/AddressSanitizer.html>. [Online; accessed 4-September-2024].
- Google, b. MemorySanitizer. <https://clang.llvm.org/docs/MemorySanitizer.html>. [Online; accessed 4-September-2024].
- Google, 2020. Fuzzbench report. URL: <https://www.fuzzbench.com/reports/2020-03-11/index.html>. accessed: 2024-09-09.
- Google, 2021. Fuzzer test suite. <https://github.com/google/fuzzer-test-suite>.

- Güler, E., Görz, P., Geretto, E., Jemmett, A., Österlund, S., Bos, H., Giuffrida, C., Holz, T., 2020. Cupid: Automatic fuzzer selection for collaborative fuzzing, in: Proceedings of the 36th Annual Computer Security Applications Conference, Association for Computing Machinery, New York, NY, USA. p. 360–372. URL: <https://doi.org/10.1145/3427228.3427266>, doi:10.1145/3427228.3427266.
- Hazimeh, A., Herrera, A., Payer, M., 2020. Magma: A ground-truth fuzzing benchmark. Proc. ACM Meas. Anal. Comput. Syst. 4. URL: <https://doi.org/10.1145/3428334>, doi:10.1145/3428334.
- Helin, A., 2018. Radamsa. <https://gitlab.com/akihe/radamsa>. Accessed: 2018.
- Holler, C., Herzig, K., Zeller, A., 2012. Fuzzing with code fragments, in: 21st USENIX Security Symposium (USENIX Security 12), USENIX Association, Bellevue, WA. pp. 445–458. URL: <https://www.usenix.org/conference/usenixsecurity12/technical-sessions/presentation/holler>.
- Hu, J., Duan, Y., Yin, H., 2024. Marco: A stochastic asynchronous concolic explorer, in: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, Association for Computing Machinery, New York, NY, USA. URL: <https://doi.org/10.1145/3597503.3623301>, doi:10.1145/3597503.3623301.
- Intel, L., 2016. Circumventing fuzzing roadblocks with compiler transformations.
- Jha, P., Scott, J., Ganeshna, J.S., Singh, M., Ganesh, V., 2024. Bertrlfuzzer: A bert and reinforcement learning based fuzzer. URL: <https://arxiv.org/abs/2305.12534>, arXiv:2305.12534.
- Karamcheti, S., Mann, G., Rosenberg, D., 2018. Adaptive grey-box fuzz-testing with thompson sampling, in: Proceedings of the 11th ACM Workshop on Artificial Intelligence and Security, Association for Computing Machinery, New York, NY, USA. p. 37–47. URL: <https://doi.org/10.1145/3270101.3270108>, doi:10.1145/3270101.3270108.
- Klees, G., Ruef, A., Cooper, B., Wei, S., Hicks, M., 2018. Evaluating fuzz testing, in: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, Association for Computing Machinery, New York, NY, USA. p. 2123–2138. URL: <https://doi.org/10.1145/3243734.3243804>, doi:10.1145/3243734.3243804.
- Komiyama, J., Honda, J., Nakagawa, H., 2019. Optimal regret analysis of thompson sampling in stochastic multi-armed bandit problem with multiple plays. URL: <https://arxiv.org/abs/1506.00779>, arXiv:1506.00779.

- Kong, Z., Li, S., Huang, H., Su, Z., 2025. Sand: Decoupling sanitization from fuzzing for low overhead. URL: <https://arxiv.org/abs/2402.16497>, arXiv:2402.16497.
- Lee, M., Cha, S., Oh, H., 2023. Learning seed-adaptive mutation strategies for greybox fuzzing, in: Proceedings of the 45th International Conference on Software Engineering, IEEE Press. p. 384–396. URL: <https://doi.org/10.1109/ICSE48619.2023.00043>, doi:10.1109/ICSE48619.2023.00043.
- Lemieux, C., Sen, K., 2018. Fairfuzz: a targeted mutation strategy for increasing greybox fuzz testing coverage, in: Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering, Association for Computing Machinery, New York, NY, USA. p. 475–485. URL: <https://doi.org/10.1145/3238147.3238176>, doi:10.1145/3238147.3238176.
- Li, P., Meng, W., Zhang, C., 2024. SDFuzz: Target states driven directed fuzzing, in: 33rd USENIX Security Symposium (USENIX Security 24), USENIX Association, Philadelphia, PA. pp. 2441–2457. URL: <https://www.usenix.org/conference/usenixsecurity24/presentation/li-penghui>.
- Li, W., Yang, H., Luo, X., Cheng, L., Cai, H., 2023. Pyrtfuzz: Detecting bugs in python runtimes via two-level collaborative fuzzing, in: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, Association for Computing Machinery, New York, NY, USA. p. 1645–1659. URL: <https://doi.org/10.1145/3576915.3623166>, doi:10.1145/3576915.3623166.
- Li, Y., Ji, S., Chen, Y., Liang, S., Lee, W.H., Chen, Y., Lyu, C., Wu, C., Beyah, R., Cheng, P., Lu, K., Wang, T., 2021. UNIFUZZ: A holistic and pragmatic Metrics-Driven platform for evaluating fuzzers, in: 30th USENIX Security Symposium (USENIX Security 21), USENIX Association. pp. 2777–2794. URL: <https://www.usenix.org/conference/usenixsecurity21/presentation/li-yuwei>.
- Liang, J., Jiang, Y., Chen, Y., Wang, M., Zhou, C., Sun, J., 2018. PafI: extend fuzzing optimizations of single mode to industrial parallel mode, in: Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Association for Computing Machinery, New York, NY, USA. p. 809–814. URL: <https://doi.org/10.1145/3236024.3275525>, doi:10.1145/3236024.3275525.
- Liang, J., Wang, M., Zhou, C., Wu, Z., Liu, J., Jiang, Y., 2024. Dodrio: Parallelizing taint analysis based fuzzing via redundancy-free scheduling, in: Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering, Association for Computing Machinery, New York, NY, USA. p. 244–254. URL: <https://doi.org/10.1145/3663529.3663844>, doi:10.1145/3663529.3663844.

- Liu, P., Yuan, W., Fu, J., Jiang, Z., Hayashi, H., Neubig, G., 2021. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. URL: <https://arxiv.org/abs/2107.13586>, arXiv:2107.13586.
- Lyu, C., Ji, S., Zhang, C., Li, Y., Lee, W.H., Song, Y., Beyah, R., 2019. MOPT: Optimized mutation scheduling for fuzzers, in: 28th USENIX Security Symposium (USENIX Security 19), USENIX Association, Santa Clara, CA. pp. 1949–1966. URL: <https://www.usenix.org/conference/usenixsecurity19/presentation/lyu>.
- Mann, H.B., Whitney, D.R., 1947. On a test of whether one of two random variables is stochastically larger than the other. The Annals of Mathematical Statistics 18, 50–60. URL: <http://www.jstor.org/stable/2236101>.
- Manès, V.J., Han, H., Han, C., Cha, S.K., Egele, M., Schwartz, E.J., Woo, M., 2021. The art, science, and engineering of fuzzing: A survey. IEEE Transactions on Software Engineering 47, 2312–2331. doi:10.1109/TSE.2019.2946563.
- Massey Jr, F.J., 1951. The kolmogorov-smirnov test for goodness of fit. Journal of the American statistical Association 46, 68–78.
- Miller, B.P., Fredriksen, L., So, B., 1990. An empirical study of the reliability of unix utilities. Commun. ACM 33, 32–44. URL: <https://doi.org/10.1145/96267.96279>, doi:10.1145/96267.96279.
- Österlund, S., Geretto, E., Jemmett, A., Güler, E., Görz, P., Holz, T., Giuffrida, C., Bos, H., 2021. Collabfuzz: A framework for collaborative fuzzing, in: Proceedings of the 14th European Workshop on Systems Security, Association for Computing Machinery, New York, NY, USA. p. 1–7. URL: <https://doi.org/10.1145/3447852.3458720>, doi:10.1145/3447852.3458720.
- Pham, V.T., Nguyen, M.D., Ta, Q.T., Murray, T., Rubinstein, B.I.P., 2022. Towards systematic and dynamic task allocation for collaborative parallel fuzzing, in: Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering, IEEE Press. p. 1337–1341. URL: <https://doi.org/10.1109/ASE51524.2021.9678810>, doi:10.1109/ASE51524.2021.9678810.
- Project, L., 2015. Libfuzzer – a library for coverage-guided fuzz testing. <https://l1vm.org/docs/LibFuzzer.html>. Accessed: 2024-02-25.
- Russo, D., Roy, B.V., Kazerouni, A., Osband, I., Wen, Z., 2020. A tutorial on thompson sampling. URL: <https://arxiv.org/abs/1707.02038>, arXiv:1707.02038.
- Scott, J., Mora, F., Ganesh, V., 2020. Banditfuzz: A reinforcement-learning based performance fuzzer for smt solvers, in: Software Verification: 12th

- International Conference, VSTTE 2020, and 13th International Workshop, NSV 2020, Los Angeles, CA, USA, July 20–21, 2020, Revised Selected Papers, Springer-Verlag, Berlin, Heidelberg. p. 68–86. URL: https://doi.org/10.1007/978-3-030-63618-0_5, doi:10.1007/978-3-030-63618-0_5.
- Sebah, P., Gourdon, X., 2002. Introduction to the gamma function. *American Journal of Scientific Research*, 2–18.
- Serebryany, K., Bruening, D., Potapenko, A., Vyukov, D., 2012. AddressSanitizer: A fast address sanity checker, in: 2012 USENIX Annual Technical Conference (USENIX ATC 12), USENIX Association, Boston, MA. pp. 309–318. URL: <https://www.usenix.org/conference/atc12/technical-sessions/presentation/serebryany>.
- Shi, W., Li, H., Yu, J., Guo, W., Xing, X., 2024. Bandfuzz: A practical framework for collaborative fuzzing with reinforcement learning, in: Proceedings of the 17th ACM/IEEE International Workshop on Search-Based and Fuzz Testing, Association for Computing Machinery, New York, NY, USA. p. 55–56. URL: <https://doi.org/10.1145/3643659.3648563>, doi:10.1145/3643659.3648563.
- Smirnov, N., 1948. Table for estimating the goodness of fit of empirical distributions. *The annals of mathematical statistics* 19, 279–281.
- Sun, Y., Xu, X., 2023. Taef: A task allocation-based ensemble fuzzing framework for optimizing the advantages of heterogeneous fuzzers. *Applied Sciences* 13. URL: <https://www.mdpi.com/2076-3417/13/24/13042>, doi:10.3390/app132413042.
- Sutton, R.S., Barto, A.G., 2018. Reinforcement learning: An introduction. MIT press.
- Thompson, W.R., 1933. On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. *Biometrika* 25, 285–294. URL: <http://www.jstor.org/stable/2332286>.
- Wang, Y., Zhang, Y., Pang, C., Li, P., Triandopoulos, N., Xu, J., 2021. Facilitating parallel fuzzing with mutually-exclusive task distribution. URL: <https://arxiv.org/abs/2109.08635>, arXiv:2109.08635.
- Wu, W.C., Nongpoh, B., Nour, M., Marcozzi, M., Bardin, S., Hauser, C., 2024. Fine-grained coverage-based fuzzing. *ACM Transactions on Software Engineering and Methodology* 33, 1–41.
- Yu, X., Liang, H., Wang, C., 2024. Multiple targets directed greybox fuzzing: From reachable to exploited, in: 2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), pp. 907–917. doi:10.1109/SANER60148.2024.00099.

- Yue, T., Tang, Y., Yu, B., Wang, P., Wang, E., 2019. Learnaff: Greybox fuzzing with knowledge enhancement. *IEEE Access* 7, 117029–117043. doi:10.1109/ACCESS.2019.2936235.
- Yun, I., Lee, S., Xu, M., Jang, Y., Kim, T., 2018. QSYM : A practical concolic execution engine tailored for hybrid fuzzing, in: 27th USENIX Security Symposium (USENIX Security 18), USENIX Association, Baltimore, MD. pp. 745–761. URL: <https://www.usenix.org/conference/usenixsecurity18/presentation/yun>.
- Zalewski, M., 2017. american fuzzy lop (2.52b). <https://lcamtuf.coredump.cx/afl/>. Accessed: 2024-02-25.
- Zhang, H., Rong, Y., He, Y., Chen, H., 2024a. Llamafuzz: Large language model enhanced greybox fuzzing. URL: <https://arxiv.org/abs/2406.07714>, arXiv:2406.07714.
- Zhang, M., Zhou, C., Liu, J., Wang, M., Liang, J., Zhu, J., Jiang, Y., 2023. Daisy: Effective fuzz driver synthesis with object usage sequence analysis, in: 2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), pp. 87–98. doi:10.1109/ICSE-SEIP58684.2023.00013.
- Zhang, Z., Chen, L., Wei, H., Shi, G., Meng, D., 2024b. Prospector: Boosting directed greybox fuzzing for large-scale target sets with iterative prioritization, in: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, Association for Computing Machinery, New York, NY, USA. p. 1351–1363. URL: <https://doi.org/10.1145/3650212.3680365>, doi:10.1145/3650212.3680365.
- Zhao, Y., Gao, L., Wan, Q., Zhao, L., 2024. Grammar-aware test case trimming for efficient hybrid fuzzing. *Journal of King Saud University - Computer and Information Sciences* 36, 101920. URL: <https://www.sciencedirect.com/science/article/pii/S1319157824000090>, doi:<https://doi.org/10.1016/j.jksuci.2024.101920>.